

Artificial Intelligence Foundations

**Google's four foundational machine learning courses,
carried over**

Prof. Dr.-Ing. Mark Schutera | August 19, 2026 · 4 blocks

Four courses, one track



After this track you can say whether a problem is an ML problem, frame it in ML terms, build and evaluate a model, and plan the project that puts it into production and keeps it there.

A new way to solve problems

Machine learning powers some of the most important technologies we use, from translation apps to autonomous vehicles. ML offers a new way to solve problems, answer complex questions, and create new content. ML can predict the weather, estimate travel times, recommend songs, auto-complete sentences, summarize articles, and generate never-seen-before images.

In basic terms, ML is the process of training a piece of software, called a **model**, to make useful predictions or generate content from data.

Predicting rainfall, two ways.

Traditional: build a physics-based representation of the atmosphere and surface, computing massive amounts of fluid dynamics equations. This is incredibly difficult.

ML: give the model enormous amounts of weather data until it learns the mathematical relationship between weather patterns that produce differing amounts of rain.

BLOCK 1

Introduction to Machine Learning

What the four kinds of ML system are, and what the five parts of a supervised one do.

1

Types of ML

Four categories, by how the system learns.

Types of ML systems

ML systems fall into one or more of the following categories based on how they learn to make predictions or generate content.

- Supervised learning
- Unsupervised learning
- Reinforcement learning
- Generative AI

The categories are not exclusive. Generative models are initially trained using an **unsupervised** approach, then sometimes trained further using supervised or reinforcement learning.

Course length: 20 minutes.

This course introduces ML concepts. It does not cover how to implement ML or work with data.

Objectives: the different types of machine learning, the key concepts of supervised ML, and how solving problems with ML differs from traditional approaches.

Supervised learning

Supervised learning models can make predictions after seeing lots of data with the correct answers and then discovering the connections between the elements in the data that produce the correct answers.

These ML systems are **supervised** in the sense that a human gives the ML system data with the known correct results. Two of the most common use cases are regression and classification.

The old-exams analogy.

This is like a student learning new material by studying old exams that contain both questions and answers. Once the student has trained on enough old exams, the student is well prepared to take a new exam.

Regression predicts a number

A regression model predicts a **numeric value**. A weather model that predicts the amount of rain, in inches or millimeters, is a regression model.

Scenario		Possible input data	Numeric prediction
Future price	house	Square footage, zip code, number of bedrooms and bathrooms, lot size, mortgage interest rate, property tax rate, construction costs, number of homes for sale in the area	The price of the home
Future ride time		Historical traffic conditions, distance from destination, weather conditions	The time to arrive at a destination

Historical traffic conditions are gathered from smartphones, traffic sensors, ride-hailing and other navigation applications.

Energy usage is measured in kilowatt-hours, which is a number, so a model predicting energy usage for commercial buildings is a regression model.

Data is the driving force

Classification predicts a category

Classification models predict the likelihood that something belongs to a category. Unlike regression models, whose output is a number, classification models output a value that states whether or not something belongs to a particular category. Classification models are used to predict if an email is spam or if a photo contains a cat.

Binary classification models output a value from a class that contains only two values, for example `rain` or `no rain`.

Multiclass classification models output a value from a class that contains more than two values, for example `rain`, `hail`, `snow`, or `sleet`.

Turning a number into classes.

To classify an airplane ticket cost as high, average, or low: find the average cost from departure to destination airport, determine the thresholds that constitute high, average, and low, then compare the predicted cost to the thresholds and output the category the value falls within.

Unsupervised learning

An unsupervised learning model aims to identify meaningful patterns in a dataset. Many unsupervised learning models rely on a technique called **clustering** to organize similar data into groups.

Clustering differs from classification because the categories aren't defined by you. An unsupervised model might cluster a weather dataset based on temperature, revealing segmentations that define the seasons. You might then attempt to name those clusters based on your understanding of the dataset.

The distinguishing question.

A supervised approach is given data that contains the correct answer. Its job is to find connections in the data that produce the correct answer.

An unsupervised approach is given data without the correct answer. Its job is to find groupings in the data. It does not know what the clusters mean.

Reinforcement learning

Reinforcement learning models make predictions by getting **rewards** or penalties based on actions performed within an environment. A reinforcement learning system generates a policy that defines the best strategy for getting the most rewards.

Reinforcement learning is used to train robots to perform tasks, like walking around a room, and software programs like AlphaGo to play the game of Go.

Generative AI

Generative AI is a class of models that creates content from user input. It can create unique images, music compositions, and jokes; it can summarize articles, explain how to perform a task, or edit a photo.

We discuss generative models by their inputs and outputs, typically written as **input type** to **output type**:

- Text-to-text, text-to-image, text-to-video
- Text-to-code, text-to-speech
- Image and text-to-image

A model can take an image as input and create an image and text as output, or take an image and text as input and create a video as output.

Generative models help businesses refine ecommerce product images, by removing distracting backgrounds or raising the quality of low-resolution images.

How generative AI works

At a high level, generative models learn patterns in data with the goal to produce new but similar data. Generative models are like:

- Comedians who learn to imitate others by observing people's behaviors and style of speaking
- Artists who learn to paint in a particular style by studying lots of paintings in that style
- Cover bands that learn to sound like a specific music group by listening to lots of music by that group

To produce unique and creative outputs, generative models are initially trained using an unsupervised approach, where the model learns to **mimic** the data it's trained on.

The model is sometimes trained further using supervised or reinforcement learning on data for tasks it might be asked to perform, for example summarize an article or edit a photo.

KEY TERMS classification model; clustering; model; policy; prediction; regression model; reinforcement learning; reward; supervised learning; training; unsupervised learning

2

Supervised concepts

Data, model, training, evaluating, inference.

Data is the driving force of ML

Data comes in the form of words and numbers stored in tables, or as the values of pixels and waveforms captured in images and audio files. We store related data in **datasets**.

Datasets are made up of individual **examples** that contain **features** and a **label**. You could think of an example as analogous to a single row in a spreadsheet. Features are the values that a supervised model uses to predict the label. The label is the answer, or the value we want the model to predict.

A weather model.

Features: latitude, longitude, temperature, humidity, cloud coverage, wind direction, atmospheric pressure. Label: rainfall amount.

Examples that contain both features and a label are called **labeled examples**. **Unlabeled examples** contain features but no label; after you create a model, the model predicts the label from the features.

Size and diversity

A dataset is characterized by its size and diversity. **Size** indicates the number of examples. **Diversity** indicates the range those examples cover. Good datasets are **both large and highly diverse**.

A large dataset doesn't guarantee sufficient diversity, and a dataset that is highly diverse doesn't guarantee sufficient examples.

A dataset might contain 100 years worth of data, but only for the month of July. Using it to predict rainfall in January would produce poor predictions.

Conversely, a dataset might cover only a few years but contain every month. It might produce poor predictions because it doesn't contain enough years to account for variability.

More features is not automatically better

A dataset can also be characterized by the number of its features. Some weather datasets might contain hundreds of features, ranging from satellite imagery to cloud coverage values. Others might contain only three or four, like humidity, atmospheric pressure, and temperature.

Datasets with more features can help a model discover additional patterns and make better predictions. However, datasets with more features don't always produce models that make better predictions, because some features might have **no causal relationship** to the label.

Predictive power.

In a used-car dataset with price as the label, make/model, year, and miles are likely among the strongest predictors. Color, height, tire size, and wheel base are not.

Model

A model is the complex collection of numbers that define the mathematical relationship from input feature patterns to output label values.

The model discovers these patterns through training.

Training

To train a model, we give the model a dataset with labeled examples. The model's goal is to work out the best solution for predicting the labels from the features. The model finds the best solution by comparing its predicted value to the label's actual value. Based on the difference between the predicted and actual values, defined as the **loss**, the model gradually updates its solution.

- The model takes in a single labeled example and provides a prediction.
- It compares its predicted value with the actual value and updates its solution.
- It repeats this for each labeled example in the dataset.

If the model predicted 1.15 inches of rain, but the actual value was .75 inches, the model modifies its solution so its prediction is closer to .75 inches.

After the model has looked at each example in the dataset, in some cases multiple times, it arrives at a solution that makes the best predictions, on average, for each of the examples.

Why large and diverse datasets produce a better model

The model gradually learns the correct relationship between the features and the label. This **gradual** understanding is also why large and diverse datasets produce a better model: the model has seen more data with a wider range of values and has refined its understanding of the relationship between the features and the label.

During training, ML practitioners can make subtle adjustments to the configurations and features the model uses. Certain features have more predictive power than others, so practitioners can select which features the model uses during training.

Suppose a weather dataset contains `time_of_day` as a feature. An ML practitioner can add or remove `time_of_day` during training to see whether the model makes better predictions with or without it.

Evaluating, then inference

We evaluate a trained model to determine how well it learned. When we evaluate a model, we use a labeled dataset, but we only give the model the dataset's **features**. We then compare the model's predictions to the label's true values. Depending on the results, we might do more training and evaluating before deploying.

Once we're satisfied, we can use the model to make predictions, called **inferences**, on unlabeled examples. In the weather app, we give the model the current conditions, like temperature, atmospheric pressure, and relative humidity, and it predicts the amount of rainfall.

If the predictions are far off.

Retrain using only the features you believe have the strongest predictive power for the label.

Retrain using a larger and more diverse dataset.

Most models require multiple rounds of training.

KEY TERMS example; feature; inference; labeled example; label; loss; prediction; training

BLOCK 2

Machine Learning Crash Course

Twelve self-contained modules: how to build regression and classification models, how to work with data, and what changes in the real world.

Twelve modules, four groups

Group	Modules
ML models	Linear regression, logistic regression, classification
Data	Numerical data, categorical data, datasets and overfitting
Advanced models	Neural networks, embeddings, large language models
Real-world ML	Production ML systems, AutoML, ML fairness

Self-contained.

Each module is self-contained, so with prior experience you can skip directly to the topics you want. If you are new to ML, complete the modules in the order shown.

Data gets three of the twelve modules. ML practitioners spend far more time evaluating, cleaning, and transforming data than building models.

3

Linear regression

Finding the relationship between features and a label.

Linear regression 80 minutes

Linear regression is a statistical technique used to find the relationship between variables. In an ML context, linear regression finds the relationship between **features** and a **label**.

Suppose we want to predict a car's fuel efficiency in miles per gallon based on how heavy the car is:

Pounds in 1000s (feature)

3.5	3.69	3.44	3.43	4.34	4.42	2.37
-----	------	------	------	------	------	------

Miles per gallon (label)

18	15	18	16	15	14	24
----	----	----	----	----	----	----

Objectives.

Explain a loss function and how it works. Define and describe how gradient descent finds the optimal model parameters. Describe how to tune hyperparameters to efficiently train a linear model.

As a car gets heavier, its miles per gallon rating generally decreases.

The linear regression equation

$$y' = b + w_1 x_1$$

In algebraic terms the model is $y = mx + b$. In ML we rename the parts:

- y' is the predicted label, the output.
- b is the **bias**, the same concept as the y-intercept. Sometimes written w_0 . It is a parameter of the model and is calculated during training.
- w_1 is the **weight** of the feature, the same concept as the slope m . Also a parameter, also calculated during training.
- x_1 is a feature, the input.

The fitted car model.

Bias is 34, where the line intersects the y-axis. Weight is -4.6 , the slope of the line. So

$$y' = 34 + (-4.6)(x_1)$$

A 4,000-pound car has a predicted fuel efficiency of 15.6 miles per gallon.

Models with multiple features

A more sophisticated model relies on multiple features, each having a separate weight:

$$y' = b + w_1x_1 + w_2x_2 + w_3x_3 + w_4x_4 + w_5x_5$$

A model that predicts gas mileage could additionally use engine displacement, acceleration, number of cylinders, and horsepower.

During training, the model calculates the weight and bias that produce the best model. The **bias and weights** are what training updates. Predictions are not updated during training, and feature values are part of the dataset.

KEY TERMS bias; feature; label; linear regression; parameter; weight

As a car's engine gets bigger, its miles per gallon rating generally decreases.

As a car's acceleration takes longer, the miles per gallon rating generally increases.

Both additional features have a linear relationship to the label.

Loss

Loss is a numerical metric that describes how wrong a model's predictions are. Loss measures the distance between the model's predictions and the actual labels. The goal of training a model is to **minimize the loss**, reducing it to its lowest possible value.

Loss focuses on the distance between the values, not the direction. If a model predicts 2 but the actual value is 5, we don't care that the loss is negative. We care that the distance between the values is 3. Thus all methods for calculating loss remove the sign.

Two ways to remove the sign.

Take the absolute value of the difference between the actual value and the prediction.

Square the difference between the actual value and the prediction.

The five types of loss

Loss type	Definition	Equation
L1 loss	Sum of the absolute values of the difference	$\sum \text{actual} - \text{predicted} $
Mean absolute error (MAE)	Average of L1 losses across N examples	$\frac{1}{N} \sum \text{actual} - \text{predicted} $
L2 loss	Sum of the squared differences	$\sum (\text{actual} - \text{predicted})^2$
Mean squared error (MSE)	Average of L2 losses across N examples	$\frac{1}{N} \sum (\text{actual} - \text{predicted})^2$
Root mean squared error (RMSE)	Square root of the MSE	$\sqrt{\frac{1}{N} \sum (\text{actual} - \text{predicted})^2}$

The functional difference between L1 and L2 is **squaring**. When the difference is large, squaring makes the loss even larger. When the difference is small, less than 1, squaring makes it even smaller.

MAE represents the average prediction error. RMSE represents the spread of the errors, and is more skewed by larger errors.

Calculating loss

For the car model $y' = 34 + (-4.6)(x_1)$, with weight -4.6 and bias 34: a 2,370-pound car is predicted to get 23.1 miles per gallon, but actually gets 24.

$$\text{prediction} = 34 + (-4.6 \cdot 2.37) = 23.1$$

$$\text{actual} = 24$$

$$\text{L2 loss} = (24 - 23.1)^2 = 0.81$$

The formula uses 2.37 because the graphs are scaled to 1000s of pounds.

When processing multiple examples at once, average the losses across all the examples, whether using MAE, MSE, or RMSE.

Choosing a loss: how should outliers count?

An outlier is a value far from the typical range. Cars are normally between 2000 and 5000 pounds and get between 8 to 50 miles per gallon. An 8,000-pound car is an outlier.

MSE. The model is closer to the outliers but further away from most of the other data points. L2 loss incurs a much higher penalty for an outlier than L1 loss.

MAE. The model is further away from the outliers but closer to most of the other data points.

Choose MSE if you want to heavily penalize large errors, or if you believe the outliers are important and indicative of true data variance the model should account for.

Choose MAE if your dataset has significant outliers you don't want to overly influence the model, or if you prefer a loss that is directly interpretable as average error magnitude.

KEY TERMS mean absolute error (MAE); mean squared error (MSE); L1; L2; loss; outlier; prediction

Gradient descent

Gradient descent is a mathematical technique that iteratively finds the weights and bias that produce the model with the lowest loss. The model begins training with randomized weights and biases near zero, then repeats the following steps.

1. Calculate the loss with the current weight and bias.
2. Determine the direction to move the weights and bias that reduces loss.
3. Move the weight and bias values a small amount in that direction.
4. Return to step one and repeat until the model can't

The **small amount** in step three is the learning rate, covered two slides on.

When a model **converges**, additional iterations don't reduce loss more, because gradient descent has found the weights and bias that nearly minimize the loss.

The math behind one step

Writing the prediction as $f_{w,b}(x) = (w \cdot x) + b$ and the actual value as y , MSE over M examples is $\frac{1}{M} \sum_{i=1}^M (f_{w,b}(x_{(i)}) - y_{(i)})^2$. Taking the derivative with respect to the weight and the bias:

$$\frac{\partial}{\partial w} \frac{1}{M} \sum_{i=1}^M (f_{w,b}(x_{(i)}) - y_{(i)})^2 = \frac{1}{M} \sum_{i=1}^M (f_{w,b}(x_{(i)}) - y_{(i)}) \cdot 2x_{(i)}$$

$$\frac{\partial}{\partial b} \frac{1}{M} \sum_{i=1}^M (f_{w,b}(x_{(i)}) - y_{(i)})^2 = \frac{1}{M} \sum_{i=1}^M (f_{w,b}(x_{(i)}) - y_{(i)}) \cdot 2$$

new weight = old weight - (small amount · weight slope)

new bias = old bias - (small amount · bias slope)

Solving with weight and bias both zero on the seven-example car dataset gives a weight slope of -119.7 and a bias slope of -34.3 .

Six iterations on the car dataset

Iteration	Weight	Bias	Loss (MSE)
1	0	0	303.71
2	1.20	0.34	170.84
3	2.05	0.59	103.17
4	2.66	0.78	68.70
5	3.09	0.91	51.13
6	3.40	1.01	42.17

Starting from weight 0 and bias 0, MSE is 303.71. With a small amount of 0.01, the first update gives weight 1.2 and bias 0.34.

In practice a model trains until it converges. Past convergence, loss fluctuates in small amounts as the model updates parameters around their lowest values, so train until the loss has stabilized.

Loss curves and convergence

A **loss curve** shows how the loss changes as the model trains. Loss is on the y-axis, iterations on the x-axis. Loss dramatically decreases during the first few iterations, then gradually decreases before flattening out around the 1,000th iteration mark.

At around the second iteration the model would not be good at making predictions because of the high amount of loss. At around the 400th, a better model. At around the 1,000th, the model has converged.

In the course's snapshots, blue loss lines run from the model to the data points and show the amount of loss. The longer the lines, the more loss there is.

Convergence and convex functions

The loss functions for linear models always produce a **convex** surface. As a result, when a linear regression model converges, we know the model has found the weights and bias that produce the lowest loss.

If we graphed the weights and bias points during gradient descent, the points would look like a ball rolling down a hill, finally stopping at the point where there's no more downward slope.

On a hypothetical miles-per-gallon dataset, a weight of -5.44 and bias of 35.94 produce the lowest loss, at 5.54 .

KEY TERMS convergence; convex function; gradient descent; iteration; loss curve

The model almost never finds the exact weight and bias values that minimize loss, but instead finds values very close to them.

The minimum loss is typically greater than 0. A loss of 0 would mean the model fits every data point exactly, which is usually a sign of overfitting.

Hyperparameters versus parameters

*Hyperparameters are values that **you** control.*

*Parameters are values that the **model** calculates during training.*

Three common hyperparameters: learning rate, batch size, and epochs. The weights and bias are parameters, part of the model itself.

Learning rate

Learning rate is a floating point number you set that influences how quickly the model converges. The model multiplies the gradient by the learning rate to determine the parameters for the next iteration.

- Too low: the model can take a very long time to converge, making only minor improvements after each iteration.
- Too high: the model never converges. It bounces around the weights and bias that minimize the loss, and loss can even increase at later iterations.

The difference between old and new parameters is proportional to the slope of the loss function. If the slope is large, the model takes a large step.

If the gradient's magnitude is 2.5 and the learning rate is 0.01, the model changes the parameter by 0.025.

The **ideal learning rate is problem-dependent**. Each model and dataset has

Batch size

Batch size is the number of examples the model processes before updating its weights and bias. When a dataset contains hundreds of thousands or millions of examples, using the full batch isn't practical.

Stochastic gradient descent (SGD) uses a single example per iteration. Given enough iterations SGD works, but is very noisy. **Noise** refers to variations during training that cause the loss to increase rather than decrease during an iteration.

Mini-batch SGD is the compromise: for N data points, batch size is any number greater than 1 and less than N . The model chooses the examples at random, averages their gradients, and updates once per iteration.

Small batch sizes behave like SGD; larger batch sizes behave like full-batch gradient descent.

A certain amount of noise can be a good thing. Noise can help a model generalize better and find the optimal weights and bias in a neural network.

By averaging more gradients together, larger batch sizes help reduce the negative effects of outliers in the data.

Epochs

An **epoch** means the model has processed every example in the training set once. Given a training set with 1,000 examples and a mini-batch size of 100, it takes 10 iterations to complete one epoch.

Batch type		When updates occur, for 1,000 examples and 20 epochs
Full batch		After all examples: 20 updates, once per epoch
Stochastic gradient descent		After a single example: 20,000 updates
Mini-batch SGD (batch 100)		After each batch: 200 updates

KEY TERMS batch size; epoch; generalize; hyperparameter; iteration; learning rate; mini-batch; mini-batch stochastic gradient

The number of epochs is a hyperparameter you set before training. In many cases you'll need to experiment with how many epochs it takes to converge.

In general, more epochs produces a better model, but also takes more time to train.

Doubling the learning rate can slow down training, by making the weights bounce around.

4

Logistic regression

Predicting the probability of a given outcome.

Logistic regression 35 minutes

In the linear regression module you explored how to construct a model to make continuous numerical predictions, such as the fuel efficiency of a car. But what if you want to build a model to answer questions like **Will it rain today?** or **Is this email spam?**

Logistic regression is designed to predict the probability of a given outcome. Practically speaking, you can use the returned probability **as is**, or converted to a binary category such as True or False, Spam or Not Spam.

If a spam-prediction model takes an email as input and outputs a value of 0.932, this implies a 93.2% probability that the email is spam.

This module uses the output as is. The classification module covers converting it into a binary category.

The sigmoid function

$$f(x) = \frac{1}{1 + e^{-x}}$$

As the input x increases, the output approaches but never reaches 1. As the input decreases, the output approaches but never reaches 0. No matter how large or small the input, the output is always greater than 0 and less than 1.

Input	-7	-3	-1	0	1	3	5	7
Output	0.001	0.047	0.269	0.50	0.731	0.952	0.993	0.999

There is a family of functions called logistic functions whose output is always between 0 and 1. The standard logistic function, also known as the **sigmoid** function, is the one above. Sigmoid means s-shaped.

e is Euler's number, a mathematical constant ≈ 2.71828 .

From linear output to probability

The linear component of a logistic regression model is

$$z = b + w_1x_1 + w_2x_2 + \dots + w_Nx_N$$

and z is then passed to the sigmoid function, yielding a probability between 0 and 1:

$$y' = \frac{1}{1 + e^{-z}}$$

A linear equation can output very big or very small values of z , but the output y' is always between 0 and 1, exclusive. A z value of -10 maps to a y' value of 0.00004.

z is called the **log-odds**, because solving the sigmoid for z gives

$$z = \ln \left(\frac{y}{1-y} \right)$$

the natural logarithm of the ratio of the probabilities of the two possible outcomes, y and $1 - y$.

The sigmoid bends the straight line into an s-shape.

Worked example

A logistic regression model with three features has bias $b = 1$ and weights $w_1 = 2, w_2 = -1, w_3 = 5$. Given inputs $x_1 = 0, x_2 = 10, x_3 = 2$:

$$\begin{aligned} z &= 1 + 2x_1 - x_2 + 5x_3 \\ &= 1 + (2)(0) - (10) + (5)(2) \\ &= 1 \end{aligned}$$

$$y = \frac{1}{1 + e^{-z}} = \frac{1}{1 + e^{-1}} = \frac{1}{1 + 0.367} = \frac{1}{1.367} = 0.731$$

Remember, the output of the sigmoid function will always be greater than 0 and less than 1.

KEY TERMS binary classification; log odds; logistic regression; sigmoid function

Why not squared loss

Squared loss works well for a linear model where the rate of change of the output values is constant. Given $y' = b + 3x_1$, each time you increment x_1 by 1, y' increases by 3.

The rate of change of a logistic regression model is **not constant**. When z is close to 0, small increases in z result in much larger changes to y than when z is a large positive or negative number.

Input	5	7	8	9	10
Logistic output	0.993	0.999	0.9997	0.9999	0.99998
Digits of precision	3	3	4	4	5

If you used squared loss for the sigmoid, then as the output got closer and closer to 0 and 1, you would need more memory to preserve the precision needed to track these values.

Log Loss

$$\text{Log Loss} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(y'_i) + (1 - y_i) \log(1 - y'_i)]$$

- N is the number of labeled examples in the dataset.
- i is the index of an example in the dataset.
- y_i is the label for the i th example. Since this is logistic regression, it must either be 0 or 1.
- y'_i is the model's prediction for the i th example, somewhere between 0 and 1.

The Log Loss equation returns the logarithm of the magnitude of the change, rather than just the distance from data to prediction.

This form calculates the **mean** Log Loss across all points. Using mean rather than total Log Loss is desirable in practice, because it decouples tuning of the batch size and the learning rate.

Regularization in logistic regression

Regularization, a mechanism for penalizing model complexity during training, is extremely important in logistic regression modeling. Without it, the **asymptotic** nature of logistic regression would keep driving loss towards 0 in cases where the model has a large number of features.

Most logistic regression models use one of two strategies to decrease model complexity:

- L2 regularization.
- Early stopping: limiting the number of training steps to halt training while loss is still decreasing.

Logistic regression models are trained using the same process as linear regression models, with two distinctions: Log Loss instead of squared loss, and regularization is critical.

KEY TERMS gradient descent; linear regression; Log Loss; logistic regression; loss function; overfitting; regularization; squared loss

5

Classification

From a probability to a category, and how to judge it.

Classification 70 minutes

In logistic regression you learned to use the sigmoid function to convert raw model output to a value between 0 and 1, for example predicting that a given email has a 75% chance of being spam. But what if your goal is not to output a probability but a **category**?

Classification is the task of predicting which of a set of classes an example belongs to. This module converts a logistic regression model that predicts a probability into a binary classification model that predicts one of two classes, and shows how to choose and calculate metrics to evaluate the quality of its predictions.

Objectives.

Determine an appropriate threshold for a binary classification model. Calculate and choose appropriate metrics to evaluate one. Interpret ROC and AUC.

The classification threshold

To convert the model's raw numerical output into one of two categories, you choose a threshold probability, called a **classification threshold**. Examples with a probability above the threshold are assigned to the **positive class**, the class you are testing for. Examples with a lower probability are assigned to the **negative class**.

While 0.5 might seem like an intuitive threshold, it's not a good idea if the cost of one type of wrong classification is greater than the other, or if the classes are imbalanced.

Suppose the model scores one email at 0.99 and another at 0.51. With a threshold of 0.5, both are spam. With a threshold of 0.95, only the 0.99 email is spam.

If only 0.01% of emails are spam, or if misfiling legitimate emails is worse than letting spam into the inbox, labelling anything at least 50% likely as spam produces undesirable results.

The confusion matrix

	Actual positive	Actual negative
Predicted positive	True positive (TP): a spam email correctly classified as spam. Sent to the spam folder.	False positive (FP): a not-spam email misclassified as spam. Legitimate mail in the spam folder.
Predicted negative	False negative (FN): a spam email misclassified as not-spam. Spam that reaches the inbox.	True negative (TN): a not-spam email correctly classified. Sent to the inbox.

Row totals give all predicted positives (TP + FP) and all predicted negatives (FN + TN), regardless of validity. Column totals give all real positives (TP + FN) and all real negatives (FP + TN), regardless of model classification.

When the total of actual positives is not close to the total of actual negatives, the dataset is **imbalanced**.

What moving the threshold does

As the threshold **increases**, the model predicts fewer positives overall, both true and false, and more negatives overall, both true and false.

A spam classifier with a threshold of **0.9999** will only label an email as spam if it considers the classification at least 99.99% likely. It is highly unlikely to mislabel a legitimate email, but also likely to miss actual spam. At a very high threshold, almost all emails, both spam and not-spam, will be classified as not-spam.

KEY TERMS binary classification; class-imbalanced dataset; classification threshold; confusion matrix; ground truth; negative class; positive class

The course's threshold widget carries three toy datasets:

Separated, where positive and negative examples are generally well differentiated.

Unseparated, where many positives score lower than negatives and vice versa.

Imbalanced, containing only a few examples of the positive class.

Accuracy, and when it lies

$$\text{Accuracy} = \frac{\text{correct classifications}}{\text{total classifications}} = \frac{TP + TN}{TP + TN + FP + FN}$$

Given a balanced dataset, accuracy can serve as a coarse-grained measure of model quality. For heavily imbalanced datasets, where one class appears very rarely, say 1% of the time, a model that predicts negative 100% of the time would score **99% on accuracy, despite being useless.**

All of the metrics in this section are calculated at a single fixed threshold, and change when the threshold changes. Very often, the user tunes the threshold to optimize one of them.

In ML, words like recall, precision, and accuracy have mathematical definitions that may differ from, or be more specific than, more commonly used meanings of the word.

Recall, false positive rate, precision

$$\text{Recall (TPR)} = \frac{\text{correctly classified actual positives}}{\text{all actual positives}} = \frac{TP}{TP + FN}$$

$$\text{FPR} = \frac{\text{incorrectly classified actual negatives}}{\text{all actual negatives}} = \frac{FP}{FP + TN}$$

$$\text{Precision} = \frac{\text{correctly classified actual positives}}{\text{everything classified as positive}} = \frac{TP}{TP + FP}$$

Precision improves as false positives decrease; recall improves as false negatives decrease. Raising the threshold decreases false positives and increases false negatives, so precision and recall often show an **inverse relationship**.

Another name for recall is **probability of detection**: what fraction of spam emails are detected by this model?

FPR is also the **probability of false alarm**: what fraction of legitimate emails were incorrectly classified as spam?

Precision: what fraction of emails classified as spam were actually spam?

Which metric, and when

Metric	Guidance
Accuracy	Use as a rough indicator of training progress for balanced datasets. For model performance, use only in combination with other metrics. Avoid for imbalanced datasets.
Recall (TPR)	Use when false negatives are more expensive than false positives.
False positive rate	Use when false positives are more expensive than false negatives.
Precision	Use when it's very important for positive predictions to be accurate.

NaN, not a number, appears when dividing by 0. When TP and FP are both 0, precision has 0 in the denominator. A model that never predicts positive would have 0 TPs and 0 FPs, so its precision is NaN. NaN can look like perfect performance but can also come from a useless model.

F1 score

$$F1 = 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}} = \frac{2TP}{2TP + FP + FN}$$

The F1 score is the **harmonic mean** of precision and recall. This metric balances the importance of both, and is preferable to accuracy for class-imbalanced datasets.

KEY TERMS accuracy; false positive rate (FPR); precision; recall

When precision and recall both have perfect scores of 1.0, F1 will also be 1.0.

When precision and recall are close in value, F1 will be close to their value. When they are far apart, F1 will be similar to whichever metric is worse.

ROC and AUC

The ROC curve is a visual representation of model performance across **all** thresholds. It is drawn by calculating TPR and FPR at every possible threshold, then graphing TPR over FPR.

The area under the ROC curve (AUC) represents the probability that the model, if given a randomly chosen positive and negative example, will rank the positive higher than the negative.

A perfect model has an AUC of 1.0. A model that does exactly as well as coin flips has a diagonal ROC from (0,0) to (1,1) and an AUC of 0.5.

The long version of the name, **receiver operating characteristic**, is a holdover from WWII radar detection.

A spam classifier with AUC of 1.0 always assigns a random spam email a higher probability of being spam than a random legitimate email. With AUC of 0.5, it does so only half the time.

Choosing a model, and a threshold, with AUC

AUC is a useful measure for comparing two models, as long as the dataset is roughly balanced. The model with greater area under the curve is generally the better one. A model with AUC below 0.5 performs **worse than chance**.

The points on a ROC curve closest to (0,1) represent the best-performing thresholds. If false alarms are highly costly, choose a threshold with a lower FPR even if TPR is reduced. If false positives are cheap and missed true positives highly costly, choose the threshold that maximizes TPR.

Fixing a worse-than-chance model.

Reverse the predictions, so 1 becomes 0 and 0 becomes 1. If a binary classifier reliably puts examples in the wrong classes more often than chance, switching the class label immediately makes its predictions better than chance without retraining.

When the dataset is imbalanced, precision-recall curves may offer a better comparative visualization.

KEY TERMS area under the ROC curve (AUC); ROC curve

Prediction bias

Prediction bias is the difference between the mean of a model's predictions and the mean of ground-truth labels in the data. A model trained on a dataset where 5% of the emails are spam should predict, on average, that 5% of the emails it classifies are spam.

If the model instead predicts 50% of the time that an email is spam, then something is wrong with the training dataset, the new dataset the model is applied to, or the model itself.

Causes.

Biases or noise in the data, including biased sampling for the training set.

Too-strong regularization, meaning the model was oversimplified and lost necessary complexity.

Bugs in the model training pipeline.

The set of features provided being insufficient for the task.

Multi-class classification

Multi-class classification is an extension of binary classification to more than two classes. If each example can only be assigned to one class, the problem can be handled as a series of binary classification problems, where one class contains one of the multiple classes and the other contains all the rest put together.

For labels A, B, and C: first build a classifier that categorizes examples as A+B versus C. Then build a second that reclassifies the A+B examples as A versus B.

If class membership isn't exclusive, so that an example can be assigned to multiple classes, this is a **multi-label** classification problem.

KEY TERMS ground truth; prediction; prediction bias; binary classification

An example of a multi-class problem is a handwriting classifier that takes an image of a handwritten digit and decides which digit, 0 to 9, is represented.

Multi-class classification is explored more deeply in the neural networks module.

6

Numerical data

The model ingests a feature vector, not the dataset.

Working with numerical data 85 minutes

This unit focuses on numerical data, meaning integers or floating-point values that behave like numbers. That is, they are additive, countable, ordered, and so on. Examples: temperature, weight, the number of deer wintering in a nature preserve.

In contrast, US postal codes, despite being five-digit or nine-digit numbers, don't behave like numbers. Postal code 40004 is not twice the quantity of postal code 20002. These numbers represent categories, specifically geographic areas, and are **categorical data**.

Three units on data.

Data is so important that this course devotes three entire units to the topic: working with numerical data, working with categorical data, and datasets, generalization, and overfitting.

Feature vectors

A model does not act directly on the rows of a dataset. The model ingests an array of floating-point values called a **feature vector**. You can think of a feature vector as the floating-point values comprising one example.

Feature vectors seldom use the dataset's raw values. You must typically process the dataset's values into representations that your model can better learn from. Would a model produce better predictions by training from the actual values rather than altered values? Surprisingly, the answer is **no**.

KEY TERMS binning; bucketing; feature engineering; feature vector; normalization; preprocessing

Determining the best way to represent raw dataset values as trainable values in the feature vector is called **feature engineering**, and it is a vital part of machine learning.

The most common techniques are **normalization**, converting numerical values into a standard range, and **binning**, converting them into buckets of ranges.

First steps: look, then measure

Before creating feature vectors, study numerical data in two ways.

Visualize. Graphs help you find anomalies or patterns hiding in the data. Look at your data graphically, either as scatter plots or histograms, not only at the beginning of the pipeline but throughout data transformations. Visualizations help you continually check your assumptions.

Evaluate statistically. Gather mean and median, standard deviation, and the values at the quartile divisions: the 0th, 25th, 50th, 75th, and 100th percentiles.

The 0th percentile is the minimum value of the column; the 100th is the maximum. The 50th percentile is the median.

Certain visualization tools are optimized for certain data formats. A tool that helps you evaluate protocol buffers may or may not help you evaluate CSV data.

Finding outliers

An outlier is a value distant from most other values in a feature or label. Outliers often cause problems in model training, so finding outliers is important.

When the delta between the 0th and 25th percentiles differs significantly from the delta between the 75th and 100th percentiles, the dataset **probably contains outliers**.

- The outlier is due to a mistake. An experimenter entered an extra zero, or an instrument malfunctioned. Generally, delete these examples.
- The outlier is legitimate. Ask: will your trained model ultimately need to infer good predictions on these outliers?

If yes, keep these outliers in your training set. Outliers in certain features sometimes mirror outliers in the label, so they could help your model make better predictions. Be careful: extreme outliers can still hurt your model.

If no, delete the outliers or apply more invasive feature engineering techniques, such as clipping.

Don't over-rely on basic statistics. Anomalies can also hide in seemingly well-balanced data.

KEY TERMS clipping; outliers

Why normalize

The goal of normalization is to transform features to be on a similar scale. Feature X spans 154 to 24,917,482; feature Y spans 5 to 22. Normalization might manipulate them so they span a similar range, perhaps 0 to 1.

- Models converge more quickly. When features have different ranges, gradient descent can bounce and slow convergence.
- Models infer better predictions.
- Avoids the **NaN trap**. When a value exceeds the floating-point precision limit, the system sets it to NaN, and other numbers eventually become NaN too.
- The model learns appropriate weights for each feature. Without scaling, the model pays too much attention to features with wide ranges.

Warning. If you normalize a feature during training, you must also normalize that feature when making predictions.

Advanced optimizers like Adagrad and Adam protect against the convergence problem by changing the effective learning rate over time.

How much disparity matters

Feature A spans -0.5 to $+0.5$; feature B spans -5.0 to $+5.0$. Both spans are narrow, but B's is ten times wider. Therefore, at the start of training, the model assumes B is ten times more important than A; training takes longer than it should; the resulting model may be suboptimal. The overall damage is relatively small, but still normalize both to the same scale.

Now feature C spans -1 to $+1$ and feature D spans $+5000$ to $+1,000,000,000$. Without normalization the model will likely be suboptimal, and training will take much longer to converge or **fail to converge entirely**.

Normalize numeric features covering distinctly different ranges, for example age and income.

Also normalize a single numeric feature that covers a wide range, such as city population.

Linear scaling

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

Linear scaling converts floating-point values from their natural range into a standard range, usually 0 to 1 or -1 to $+1$. For a feature `quantity` whose natural range spans 100 to 900, a value of 300 normalizes as follows.

```
x' = (300 - 100) / (900 - 100)
x' = 200 / 800
x' = 0.25
```

Good when the lower and upper bounds don't change much over time, the feature contains few or no extreme outliers, and the feature is approximately uniformly distributed across its range.

Human age qualifies: bounds are roughly 0 to 100, only about 0.3% of the population is over 100, and a large dataset should contain sufficient examples of all ages.

`net_worth` does not: many outliers, values not uniformly distributed.

Z-score scaling

$$x' = \frac{x - \mu}{\sigma}$$

A Z-score is the number of standard deviations a value is from the mean. A value 2 standard deviations greater than the mean has a Z-score of +2.0.

```
mean = 100, standard deviation = 20  
original value = 130
```

```
Z-score = (130 - 100) / 20  
Z-score = 30 / 20  
Z-score = +1.5
```

Z-score is a good choice when the data follows a normal distribution or something like one. It is typically a better choice than linear scaling, because most real-world features do not meet all the criteria for linear scaling.

Adult height over ten million women qualifies: it conforms to a normal distribution, and ten million examples implies enough outliers for the model to learn patterns on very high or very low Z-scores.

How rare is a Z-score of 4?

In a classic normal distribution:

- At least 68.27% of data has a Z-score between -1.0 and $+1.0$.
- At least 95.45% between -2.0 and $+2.0$.
- At least 99.73% between -3.0 and $+3.0$.
- At least 99.994% between -4.0 and $+4.0$.

A feature with one billion examples conforming to a classic normal distribution could still have as many as **60,000** examples outside the range -4.0 to $+4.0$.

So are those points truly outliers? Outliers is a concept without a strict definition, so no one can say for sure.

Some distributions are normal within the bulk of their range but contain extreme outliers. Nearly all points in `net_worth` might fit within 3 standard deviations, but a few could be hundreds away. Combine Z-score scaling with clipping.

Log scaling

$$x' = \ln(x)$$

Log scaling is helpful when the data conforms to a **power law** distribution: low values of X have very high values of Y, and as X increases, Y quickly decreases.

~4.6 = $\ln(100)$
~13.8 = $\ln(1,000,000)$

The log of 1,000,000 is only about three times larger than the log of 100.

Movie ratings: a few movies have lots of user ratings, most have very few.

Book sales: most published books sell one or two hundred copies, some sell thousands, only a few bestsellers sell more than a million.

A linear model on raw sales would have to find something about a million-selling book cover that is 10,000 times more powerful than a 100-selling cover. You could imagine it being about three times more powerful.

Clipping

Clipping caps the value of outliers to a specific maximum value. Clipping is a strange idea, and yet, it can be very effective.

For a feature `roomsPerPerson`, over 99% of values conform to a normal distribution with mean about 1.8 and standard deviation about 0.7, but a few outliers reach all the way out to 17. Capping at 4.0 does not mean the model ignores values greater than 4.0. It means all values that were greater than 4.0 **now become 4.0**. That explains the peculiar hill at 4.0, and the scaled feature set is now more useful than the original data.

You can also clip after applying other normalization. If you use Z-score scaling but a few outliers have absolute values far greater than 3, clip Z-scores greater than 3 to exactly 3 and less than -3 to exactly -3 .

Clipping prevents your model from overindexing on unimportant data. However, some outliers are actually important, so clip values carefully.

The four techniques, by distribution shape

Technique	Formula	When to use
Linear scaling	$x' = \frac{x - x_{min}}{x_{max} - x_{min}}$	Mostly uniformly distributed across range. Flat-shaped.
Z-score scaling	$x' = \frac{x - \mu}{\sigma}$	Normally distributed, peak close to mean. Bell-shaped.
Log scaling	$x' = \log(x)$	Heavily skewed on at least one tail. Heavy-tail-shaped.
Clipping	If $x > max$, set $x' = max$; if $x < min$, set $x' = min$	Contains extreme outliers.

KEY TERMS clipping; NaN trap; normalization; scaling; Z-score normalization

The best normalization technique is one that works well in practice, so try new ideas if you think they'll work well on your feature distribution.

Mistakes get deleted, legitimate outliers get clipped. For a data centre temperature feature that is normally 15 to 30, clip the legitimate values between 31 and 45, but delete the every-1000th values of 1,000.

Binning

Binning, also called bucketing, groups numerical subranges into bins. In many cases binning turns numerical data into categorical data. A feature X spanning 15 to 425 might become five bins.

Bin	Range	Feature vector
1	15–34	[1.0, 0.0, 0.0, 0.0, 0.0]
2	35–117	[0.0, 1.0, 0.0, 0.0, 0.0]
3	118–279	[0.0, 0.0, 1.0, 0.0, 0.0]
4	280–392	[0.0, 0.0, 0.0, 1.0, 0.0]
5	393–425	[0.0, 0.0, 0.0, 0.0, 1.0]

A model trained on these bins will react no differently to X values of 17 and 29, since both are in bin 1.

Even though X is a single column, binning causes a model to treat it as **five separate features**, and the model learns separate weights for each bin.

When binning beats scaling

Binning is a good alternative to scaling or clipping when the overall linear relationship between the feature and the label is weak or nonexistent, or when the feature values are clustered.

Predicting the number of shoppers from outside temperature: the plot shows three clusters, at 4 to 11, 12 to 26, and 27 to 36 degrees. Represented as one raw feature, a temperature of 35.0 would have five times the influence of a temperature of 7.0, but the plot shows **no linear relationship** at all.

Binning can feel counterintuitive, given that the model treats the values 37 and 115 identically. But when a feature appears more clumpy than linear, binning is a much better way to represent the data.

More bins is usually a bad idea. A model can only learn the association between a bin and a label if there are enough examples in that bin, and you should minimize the number of features in a model.

Quantile bucketing

Quantile bucketing creates boundaries such that the number of examples in each bucket is exactly or nearly equal. Quantile bucketing mostly **hides the outliers**.

With ten equally spaced 10,000-dollar car-price buckets, the bucket from 0 to 10,000 contains dozens of examples but the bucket from 50,000 to 60,000 contains only 5. The model has enough examples to train on the first but not the second. Quantile buckets instead give the first bucket 0 to 4,000, the second 4,001 to 6,000, and the sixth 25,001 to 60,000.

Bucketing with equal intervals works for many data distributions. For skewed data, try quantile bucketing.

Equal intervals give extra information space to the long tail while compacting the large torso into a single bucket. Quantile buckets give extra space to the large torso while compacting the long tail.

KEY TERMS binning; bucketing; clipping; feature; feature engineering; feature vector; label; scaling

Scrubbing

Apple trees produce a mixture of great fruit and wormy messes. Yet the apples in high-end grocery stores display 100% perfect fruit. As an ML engineer, you'll spend enormous amounts of your time tossing out bad examples and cleaning up the salvageable ones. **Even a few bad apples can spoil a large dataset.**

Problem category	Example
Omitted values	A census taker fails to record a resident's age
Duplicate examples	A server uploads the same logs twice
Out-of-range feature values	A human accidentally types an extra digit
Bad labels	A human evaluator mislabels an oak tree as a maple

You can write a program or script to detect omitted values, duplicate examples, and out-of-range feature values.

When labels are generated by multiple people, statistically determine whether each rater generated equivalent sets of labels. Perhaps one rater was a harsher grader, or used different grading criteria.

Qualities of good numerical features

Clearly named. Each feature should have a clear, sensible, and obvious meaning to any human on the project.

```
not recommended: house_age: 851472000  
recommended:      house_age_years: 27
```

Checked or tested before training. Sometimes bad data, rather than bad engineering choices, causes unclear values.

```
not recommended: user_age_in_years: 224  
recommended:      user_age_in_years: 24
```

Although your co-workers will rebel against confusing feature and label names, the model won't care, assuming you normalize values properly.

Check your data.

Sensible: no magic values

A magic value is a purposeful discontinuity in an otherwise continuous feature. A `watch_time_in_seconds` of `-1` would force the model to figure out what it means to watch a movie **backwards in time**.

```
not recommended:
  watch_time_in_seconds: -1

recommended:
  watch_time_in_seconds: 4.82
  is_watch_time_in_seconds_defined = True

  watch_time_in_seconds: 0
  is_watch_time_in_seconds_defined = False
```

That is the way to handle a continuous feature with missing values: a separate Boolean feature indicating whether a value is supplied.

For a discrete numerical feature like `product_category`, signify the missing value using a new value in the finite set:

```
{0: 'electronics', 1:
'books', 2: 'clothing', 3:
'missing_category'}
```

The model then learns different weights for each value, including missing

Polynomial transforms

Sometimes domain knowledge suggests that one variable is related to the square, cube, or other power of another. It's possible to keep the linear equation and allow nonlinearity by defining a new term:

$$x_2 = x_1^2 \quad \Rightarrow \quad y = b + w_1x_1 + w_2x_2$$

This synthetic feature, called a polynomial transform, is treated like any other feature. This can still be treated like a linear regression problem, and the weights determined through gradient descent as usual, despite containing a hidden squared term.

Many relationships in the physical world are related to squared terms, including acceleration due to gravity, the attenuation of light or sound over distance, and elastic potential energy.

If you transform a feature in a way that changes its scale, consider normalizing it as well.

The related concept in categorical data is the **feature cross**, which more frequently synthesizes two different features.

KEY TERMS categorical data; feature; feature cross; gradient descent; linear regression; synthetic feature; weight

Best practices for numerical data

- Remember that your model interacts with the data in the feature vector, not the data in the dataset.
- Normalize most numerical features. If your first strategy doesn't succeed, consider a different one.
- Binning is sometimes better than normalizing.
- Write verification tests to validate what your data should look like. The absolute value of latitude should never exceed 90. If your data is restricted to Florida, latitudes should fall between 24 and 31.
- Visualize your data with scatter plots and histograms. Look for anomalies.
- Gather statistics not only on the entire dataset but also on smaller subsets, because aggregate statistics sometimes obscure problems.
- Document all your data transformations

A model's health is determined by its data.

Feed your model healthy data and it will thrive; feed your model junk and its predictions will be worthless.

Data is your **most valuable** resource, so treat it with care.

7

Categorical data

Encoding a specific set of possible values.

Working with categorical data 50 minutes

Categorical data has a specific set of possible values: the different species of animals in a national park, the names of streets in a particular city, whether or not an email is spam, the colors that house exteriors are painted, and binned numbers.

True numerical data can be meaningfully **multiplied**. All else being equal, a house of 200 square meters should be roughly twice as valuable as an identical house of 100 square meters.

Often you should represent features that contain integer values as categorical data instead of numerical data.

If you represent postal codes numerically, you're telling the model to treat postal code 20004 as twice as large a signal as postal code 10002. Representing them as categorical data lets the model weight each individual postal code separately.

Vocabulary and index numbers

When a categorical feature has a low number of possible categories, encode it as a **vocabulary**. The model treats each possible categorical value as a separate feature and learns different weights for each.

Feature name	Categories	Sample categories
snowed_today	2	True, False
skill_level	3	Beginner, Practitioner, Expert
season	4	Winter, Spring, Summer, Autumn
day_of_week	7	Monday, Tuesday, Wednesday
planet	8	Mercury, Venus, Earth

Machine learning models can only manipulate floating-point numbers, so you must convert each string to a unique index number.

But if the data is left as indexed integers and loaded into a model, the model would treat the indexed values as continuous floating-point numbers. It would consider purple six times more likely than orange.

One-hot encoding

Each category is represented by a vector of N elements, where N is the number of categories. Exactly one of the elements has the value 1.0; all the remaining elements have the value 0.0.

Feature	Red	Orange	Blue	Yellow	Green	Black	Purple	Brown
"Red"	1	0	0	0	0	0	0	0
"Orange"	0	1	0	0	0	0	0	0
"Blue"	0	0	1	0	0	0	0	0
"Yellow"	0	0	0	1	0	0	0	0
"Green"	0	0	0	0	1	0	0	0

It is the one-hot vector, not the string or the index number, that gets passed to the feature vector.

The model learns a **separate weight** for each element.

In a true one-hot encoding, only one element has the value 1.0. In a variant known as **multi-hot** encoding, multiple values can be 1.0

Sparse representation

A feature whose values are predominantly zero is a **sparse feature**. Sparse representation means storing the position of the 1.0. The one-hot vector for "Blue" is

```
[0, 0, 1, 0, 0, 0, 0, 0]
```

and since the 1 is in position 2, counting from 0, the sparse representation is simply 2.

The sparse representation consumes far less memory than the eight-element one-hot vector. Importantly, the model must **train on the one-hot vector**, not the sparse representation.

The sparse representation of a multi-hot encoding stores the positions of all the nonzero elements. A car that is both "Blue" and "Black" is 2, 5.

Outliers and high dimensionality

Rather than giving each rarely used outlier colour such as "Mauve" its own category, lump them into a single catch-all category called **out-of-vocabulary** (OOV). The system learns a single weight for that outlier bucket.

Feature name	Categories	Sample
words_in_english	~500,000	"happy", "walking"
US_postal_codes	~42,000	"02114", "90301"
last_names_in_Germany	~850,000	"Schmidt", "Schneider"

When the number of categories is high, one-hot encoding is usually a bad choice. **Embeddings** are usually much better: the model trains faster and infers predictions more quickly, so it has lower latency.

Hashing, the hashing trick, is a less common way to reduce dimensions.

Hashing

Hashing maps a category to a small integer, the number of the bucket that will hold that category.

1. Set the number of bins to N , less than the total number of categories. Say $N = 100$.
2. Choose a hash function.
3. Pass each category through it: say 89237.
4. Bin index = hash mod N : $89237 \% 100 = 37$
5. One-hot encode each bin with the new index.

A machine learning model **cannot** train directly on raw string values like "Red" and "Black". During training, a model can only manipulate floating-point numbers.

KEY TERMS categorical data; dimensions; discrete feature; feature cross; feature vector; hashing; one-hot encoding; sparse feature; sparse representation

Who labels the categories

Numerical data is often recorded by scientific instruments. Categorical data is often categorized by human beings or by ML models. Who decides on categories and labels, and how they make those decisions, affects the reliability and usefulness of that data.

Gold labels are manually labeled by human beings, and are considered more desirable due to relatively better data quality. This doesn't mean any set of human-labeled data is high quality. Human errors, bias, and malice can be introduced at collection or during cleaning.

Silver labels are machine-labeled, and can vary widely in quality.

Any two human beings may label the same example differently. The difference between raters' decisions is **inter-rater agreement**. Measure it with Cohen's kappa, intra-class correlation, or Krippendorff's alpha.

Check machine labels not only for accuracy and biases but also for violations of common sense, reality, and intention: a computer-vision model mislabelling a chihuahua as a muffin.

Feature crosses

Feature crosses are created by taking the Cartesian product of two or more categorical or bucketed features. Like polynomial transforms, feature crosses allow linear models to handle nonlinearities. They also encode **interactions** between features.

For a leaf dataset with edges in {smooth, toothed, lobed} and arrangement in {opposite, alternate}, the cross is

{Smooth_Opposite, Smooth_Alternate, Toothed_Opposite, Toothed_Alternate, Lobed_Opposite, Lobed_Alternate}

where each term is the product of the base feature values. A leaf with a lobed edge and alternate arrangement gives {0, 0, 0, 0, 0, 1}.

KEY TERMS feature cross; neural network; inter-rater agreement

Polynomial transforms typically combine numerical data; feature crosses combine categorical data.

Be careful. Crossing two sparse features produces an even sparser new feature. A 100-element sparse feature crossed with a 200-element sparse feature yields a 20,000-element sparse feature.

8

Overfitting

Making good predictions on data the model has never seen.

Datasets, generalization, and overfitting 105 minutes

Two leading questions open this module.

If you had to prioritize improving one area in your ML project, which would have the most impact? Improving the quality of your dataset. **Data trumps all.** The quality and size of the dataset matters much more than which shiny algorithm you use to build your model.

How much time do you typically spend on data preparation and transformation? More than half the project time. Typically, 80% of the time on a machine learning project is spent constructing datasets and transforming data.

A better loss function can help a model train faster, but it's still a distant second to the quality of the dataset.

This module covers the characteristics of ML datasets, and how to prepare your data to ensure high-quality results when training and evaluating your model.

Quantity of data

As a rough rule of thumb, your model should train on at least an **order of magnitude** (or two) more examples than trainable parameters. However, good models generally train on substantially more examples than that.

Models trained on large datasets with few features generally outperform models trained on small datasets with a lot of features. Google has historically had great success training simple models on large datasets.

For some relatively simple problems, a few dozen examples might be sufficient. For other problems, a trillion examples might be insufficient.

It's possible to get good results from a small dataset if you are adapting an existing model already trained on large quantities of data from the same schema.

Quality and reliability

A high-quality dataset helps your model accomplish its goal. A low quality dataset inhibits your model from accomplishing its goal.

Reliability refers to the degree to which you can trust your data. In measuring it, determine:

- How common are label errors? If your data is labeled by humans, how often did your raters make mistakes?
- Are your features noisy? Be realistic: you can't purge all noise. GPS measurements always fluctuate a little, week to week.
- Is the data properly filtered for your problem? Should your dataset

Common causes of unreliable data.

Omitted values. Duplicate examples. Bad feature values. Bad labels. Bad sections of data, for example a very reliable feature except for the one day when the network kept crashing.

Use **automation** to flag unreliable data: unit tests that rely on an external formal data schema can flag values outside a defined range.

Complete versus incomplete examples

In a perfect world, each example contains a value for each feature. Real-world examples are often incomplete. **Don't train a model on incomplete examples.** Instead:

- Delete incomplete examples.
- Impute missing values, converting the incomplete example to a complete one by providing well-reasoned guesses.

If you can't decide whether to delete or impute, consider building two datasets, one formed by deleting and the other by imputing, then determine which trains the better model.

Imputation is the process of generating well-reasoned data, not random or deceptive data. Good imputation can improve your model; bad imputation can hurt it.

A good dataset tells the model which values are imputed: add a Boolean column such as `temperature_is_imputed`. The model then gradually learns to trust imputed examples less.

One common algorithm uses the mean or median. With Z-scores, the imputed value is typically 0.

Direct versus proxy labels

Direct labels are identical to the prediction your model is trying to make. A column named `bicycle_owner` is a direct label for a model that predicts whether a person owns a bicycle.

Proxy labels are similar but not identical. A person subscribing to *Bicycle Bizarre* magazine probably, but not definitely, owns a bicycle.

Direct labels are generally better. Proxy labels are always a compromise, an imperfect approximation. Models that use proxy labels are only as useful as the **connection** between the proxy label and the prediction.

Sometimes a direct label exists but can't easily be represented as a floating-point number. In that case, use a proxy label.

`recently bought a bicycle` is a relatively good proxy for bicycle ownership. Like all proxy labels it is imperfect: the person buying an item isn't always the person using it. People sometimes buy bicycles as a gift.

Human-generated data

Advantages. Human raters can perform a wide range of tasks that even sophisticated ML models find difficult. The process forces the owner of the dataset to develop clear and consistent criteria.

Disadvantages. You typically pay human raters, so human-generated data can be expensive. To err is human, so multiple raters might have to evaluate the same data.

Always double-check your human raters. Label 1000 examples yourself and see how your results match. If discrepancies surface, **don't assume your ratings are the correct ones**, especially if a value judgment is involved.

KEY TERMS label; feature vector

Questions to settle first.

How skilled must your raters be?
Must they know a specific language? Do you need linguists for dialogue or NLP applications?
How many labeled examples do you need, and how soon?
What's your budget?

Class-imbalanced datasets

In a class-balanced dataset, the number of positive and negative classes is about equal. In a class-imbalanced dataset, one label is considerably more common. **In the real world, class-imbalanced datasets are far more common.**

In a dataset of credit card transactions, fraudulent purchases might make up less than 0.1% of examples. In a medical diagnosis dataset, patients with a rare virus might be less than 0.01%. The more common label is the **majority class**; the less common one is the **minority class**.

With 200 majority labels and 2 minority labels, a batch size of 20 means most batches contain no minority examples at all. A batch size of 100 averages only one, which is insufficient for proper training.

Accuracy is usually a poor metric for assessing a model trained on a class-imbalanced dataset.

Downsample and upweight the majority class

During training, a model should learn two things: what each class looks like, and how common each class is. Standard training conflates these two goals. This two-step technique separates them.

Step 1: downsample. Train on a disproportionately low percentage of majority class examples. Downsampling a 99%/1% dataset by a factor of 25 gives a training set that is 80% majority and 20% minority.

Step 2: upweight. Treat the loss on a majority class example more harshly, by the same factor. When the model mistakenly

Many students read this section and say some variant of "that just can't be right". Be warned that downsampling and upweighting the majority class is somewhat counterintuitive.

Downsampling introduces a prediction bias, by showing the model an artificial world where the classes are more balanced than in the real world. Upweighting corrects it.

Benefits: a better model, which knows both the connection between features and labels and

Training, validation, and test sets

Splitting into just training and test is a decent idea, but a better approach is three subsets. The **validation set** performs the initial testing on the model as it is being trained.

A typical split is 70% training, 15% validation, 15% test. Use the validation set to evaluate results from the training set. After repeated use of the validation set suggests your model is making good predictions, use the test set to double-check your model.

Tweak model means adjusting anything about the model, from changing the learning rate, to adding or removing features, to designing a completely new model from scratch.

Why not just two sets.

Doing many rounds of tuning against the test set might cause the model to implicitly fit the peculiarities of the test set. Like a teacher teaching to the test, the model inadvertently fits the test set.

When you transform a feature in your training set, you must make the same transformation in the validation set, test set, and real-world dataset.

Test sets wear out

Even with the three-way split, test sets and validation sets still wear out with repeated use. The more you use the same data to make decisions about hyperparameter settings or other model improvements, the less confidence that the model will make good predictions on new data. Collect more data to refresh them. **Starting anew is a great reset.**

A good test or validation set is large enough to yield statistically significant results, representative of the dataset as a whole, representative of the real-world data the model will encounter, and has **zero examples duplicated in the training set.**

KEY TERMS test set; training set; validation set

A spam model achieves 99% precision on both training and test sets. You'd expect lower precision on the test set. It turns out many test examples are duplicates of training examples: duplicate entries were never scrubbed before splitting. You've inadvertently trained on some of your test data.

Dividing examples into train, test, and validation sets is a zero-sum game. This is the central trade-off.

Overfitting

Overfitting means creating a model that matches the training set so closely that the model fails to make correct predictions on new data. An overfit model is analogous to an invention that **performs well in the lab but is worthless in the real world**.

An underfit model doesn't even make good predictions on the training data. If an overfit model is like a product that performs well in the lab but poorly in the real world, an underfit model is like a product that doesn't even do well in the lab.

Generalization is the opposite of overfitting. Your goal is to create a model that generalizes well to new data.

In the course's forest example, a complex shape separates all but two of the training trees. If we think of the shapes as a model, this is a fantastic model. Or is it? On the test set the same model miscategorizes many trees.

Overfitting is a common problem in machine learning, not an academic hypothetical.

Detecting overfitting

A **loss curve** plots a model's loss against the number of training iterations. A graph that shows two or more loss curves is a **generalization curve**.

When the two curves behave similarly at first and then diverge, so that loss declines or holds steady for the training set but **increases** for the validation set, that suggests overfitting. In contrast, a generalization curve for a well-fit model shows two loss curves that have similar shapes.

Broadly, overfitting is caused by one or both of: the training set doesn't adequately represent real life data, or the model is too complex.

Generalization conditions.

Examples must be independently and identically distributed, a fancy way of saying your examples can't influence each other.

The dataset is stationary, meaning it doesn't change significantly over time.

The dataset partitions have the same distribution.

When the conditions break

Not stationary. A streaming service predicting the popularity of new shows for the next three years, trained on ten years of data, will struggle: viewer tastes change in ways that past behavior can't predict.

Not independent. A model recommending the ideal date to buy a train ticket exhibits low loss on validation and test sets but sometimes makes terrible real-world predictions, because of a **feedback loop**. Riders who follow the recommendation buy at 8:30 on July 8; at 9:00 the company raises prices because fewer seats remain. Riders using the model's recommendation have altered prices.

To make partitions statistically similar, shuffle the examples in the dataset extensively before partitioning them.

Sorting examples from earliest to most recent makes the partitions less similar if the examples are not stationary.

Given enough examples, the law of averages does **not** naturally ensure similar distributions.

KEY TERMS feedback loop; generalization; generalization curve; independently and identically distributed; loss curve; overfitting;

Occam's razor

Replace the complex forest model with a ridiculously simple one, a straight line, and it **generalizes better** on new data.

Simplicity has been beating complexity for a long time. The preference for simplicity dates back to ancient Greece. Centuries later, a fourteenth-century friar named William of Occam formalized it in a philosophy known as Occam's razor. This philosophy remains an essential underlying principle of many sciences, including machine learning.

Complex models typically outperform simple models on the training set. However, simple models typically outperform complex models on the test set, which is more important.

How many features to start with?

Pick 1 to 3 that seem to have strong predictive power. Every new feature adds a dimension. When the dimensionality increases, the volume of the space increases so fast that the available training data become sparse. This is the **curse of dimensionality**.

The two most famous physics formulas of all time, $F = ma$ and $E = mc^2$, each involve only three variables.

Loss plus complexity

Machine learning models must simultaneously meet two conflicting goals: fit data well, and fit data as simply as possible. One approach is to penalize complex models, forcing the model to become simpler during training. That is **regularization**.

$$\text{minimize}(\text{loss}) \longrightarrow \text{minimize}(\text{loss} + \text{complexity})$$

Loss and complexity are typically inversely related. As complexity increases, loss decreases. Find a reasonable middle ground where the model makes good predictions on both the training data and real-world data.

A regularization analogy.

Suppose every student in a lecture hall had a buzzer that annoyed the professor, pressed whenever the lecture became too complicated. The professor would complain, "when I simplify, I'm not being precise enough". The students would counter, "the only goal is to explain it simply enough that I understand it". Gradually the buzzers would train the professor to give an appropriately simple lecture.

L2 regularization

$$L_2 \text{ regularization} = w_1^2 + w_2^2 + \dots + w_n^2$$

	Value	Squared value
w_1	0.2	0.04
w_2	-0.5	0.25
w_3	5.0	25.0
w_4	-1.2	1.44
w_5	0.3	0.09
w_6	-0.1	0.01
		26.83 total

Weights close to zero don't affect L2 regularization much, but large weights have a huge impact. A single weight, w_3 , contributes about 93% of the total complexity. The other five contribute only about 7%.

L2 regularization encourages weights toward 0, but **never** pushes weights all the way to zero. Consequently, all features still contribute something to the model.

Regularization rate (lambda)

minimize(loss + λ complexity)

A high regularization rate strengthens the influence of regularization, reducing the chances of overfitting. It tends to produce a histogram of model weights that is a normal distribution with a mean weight of 0.

A low regularization rate lowers the influence of regularization, increasing the chances of overfitting, and tends to produce a flat distribution of weights.

Setting the regularization rate to zero removes regularization completely. Training then focuses exclusively on minimizing loss, which poses the highest possible overfitting risk.

The ideal regularization rate produces a model that generalizes well to new, previously unseen data. That ideal value is data-dependent, so you must do some tuning.

Early stopping, and the equilibrium

Early stopping is a regularization method that doesn't involve a calculation of complexity. It simply means ending training before the model fully converges, for example when the loss curve for the validation set starts to increase. It usually increases training loss but can decrease test loss. It is a quick, but rarely optimal, form of regularization.

Learning rate and regularization rate tend to move weights in **opposite directions**. A high learning rate often pulls weights away from zero; a high regularization rate pushes weights towards zero.

If the regularization rate is high with respect to the learning rate, the weak weights tend to produce a model that makes poor predictions.

If the learning rate is high with respect to the regularization rate, the strong weights tend to produce an overfit model.

Worst of all, once you find that elusive balance, you may ultimately have to change the learning rate. And then you'll again have to find the ideal regularization rate.

Reading four broken loss curves

Symptom

What to do

Loss oscillates erratically, never flattening

Check your data against a data schema and remove bad examples. Reduce the learning rate. Reduce the training set to a tiny number of trustworthy examples, then gradually add more.

Loss decreases, then a sharp jump up

The input data contains one or more NaNs, for example from a division by zero. Or a batch contains a burst of outliers, due to improper shuffling.

Training loss converges, validation loss rises

The model is overfitting. Make the model simpler, increase the regularization rate, or ensure the training and test sets are statistically equivalent.

Loss converges, then re-

The training set is not shuffled well. 100 im-

Avoid increasing the learning rate when a model's learning curve indicates a problem.

Increasing the number of examples in the training set is a tempting idea for the oscillating curve, but is extremely unlikely to fix the problem.

A very high regularization rate could prevent a model from converging, but won't cause a sharp jump.

9

Neural networks

Learning the feature crosses for you.

Neural networks 75 minutes

Nonlinear means that you can't accurately predict a label with a model of the form $b + w_1x_1 + w_2x_2$. In other words, the decision surface is not a line.

Adding a feature cross $x_3 = x_1x_2$ lets a linear model learn a hyperbolic shape. But determining the correct feature crosses takes effort and experimentation. What if you didn't have to do all that yourself?

Neural networks are a family of model architectures designed to find nonlinear patterns in data. During training, the model **automatically learns** the optimal feature crosses to perform on the input data to minimize loss.

Objectives.

Explain the motivation for building neural networks. Define nodes, hidden layers, and activation functions. Step through the inference process. Build a high-level intuition of backpropagation. Explain one-vs.-all and one-vs.-one multi-class classification.

Nodes and hidden layers

Start with a linear model $y' = b + w_1x_1 + w_2x_2 + w_3x_3$: three input nodes and one output node. Adjusting the weights and bias does not change the overall mathematical relationship. It is still a linear model.

Additional layers between the input layer and the output layer are called **hidden layers**, and the nodes in these layers are called **neurons**. The value of each neuron is calculated the same way as the output of a linear model: the sum of the product of each of its inputs and a unique weight parameter, plus the bias.

KEY TERMS bias; hidden layer; input layer; linear model; model; neural network; neuron; nonlinear; parameter; weight

Counting parameters.

One hidden layer of 4 neurons over 3 inputs: 4 parameters per neuron, 3 weights and a bias, so 16. Then 5 more for the output: 4 weights and a bias. In total **21** parameters.

Can this model learn nonlinearities? **No**. All the calculations are linear, and linear calculations performed on the output of linear calculations are also linear.

Activation functions

We need some way to insert nonlinear mathematical operations into a model. An **activation function** is a nonlinear transform of a neuron's output value before the value is passed as input to the next layer.

Now that we've added an activation function, adding layers has more impact. Stacking nonlinearities on nonlinearities lets us model very complicated relationships between the inputs and the predicted outputs. Each layer is effectively learning a more complex, higher-level function over the raw inputs.

We have already applied a nonlinear operation to a linear model's output: in logistic regression, the sigmoid function.

Applying the same principle here, each node's output becomes a sigmoid transform of the linear combination of the nodes in the previous layer, squished between 0 and 1.

The three common activation functions

sigmoid $F(x) = \frac{1}{1 + e^{-x}}$ range 0 to 1

tanh $F(x) = \tanh(x)$ range -1 to 1

ReLU $F(x) = \max(0, x)$ 0 below zero, x above

ReLU often works a little better than a smooth function like sigmoid or tanh, because it is less susceptible to the vanishing gradient problem during training. ReLU is also significantly easier to compute.

The term sigmoid is often used more generally to refer to any S-shaped function. A more technically precise term for $F(x) = \frac{1}{1+e^{-x}}$ is **logistic function**.

In practice, any mathematical function can serve as an activation function. With σ the activation function, the value of a node is $\sigma(w \cdot x + b)$.

Keras supports many activation functions out of the box. Still, start with ReLU.

What a neural network is

The model now has all the standard components of what people usually mean when they refer to a neural network.

- A set of nodes, analogous to neurons, organized in layers.
- A set of learned weights and biases representing the connections between each layer and the layer beneath it.
- An activation function that transforms the output of each node in a layer. Different layers may have different activation functions.

KEY TERMS activation function; sigmoid function; vanishing gradient problem

A caveat.

Neural networks aren't necessarily always better than feature crosses, but they do offer a **flexible alternative** that works well in many cases.

Backpropagation

Backpropagation is the most common training algorithm for neural networks. It makes gradient descent feasible for multi-layer neural networks. Many ML code libraries handle backpropagation automatically, so you don't need to perform any of the underlying calculations yourself.

Vanishing gradients. The gradients for the lower layers, those closer to the input, can become very small. In deep networks, computing these gradients can involve taking the product of many small terms. When the values approach 0, the gradients are said to **vanish**, and those layers train very slowly or not at all. ReLU can help prevent this.

Exploding gradients.

If the weights are very large, the gradients for the lower layers involve products of many large terms and get too large to converge. Batch normalization can help, as can lowering the learning rate.

The backpropagation algorithm uses the calculus concept of a gradient. Understanding and debugging these issues usually requires some background in calculus.

Dead ReLU units, and dropout

Once the weighted sum for a ReLU unit falls below 0, the unit can get stuck. It outputs 0, contributing nothing to the network's output, and gradients can no longer flow through it. With a source of gradients cut off, the input may never change enough to bring the weighted sum back above 0.

Lowering the learning rate can help keep ReLU units from dying.

Dropout regularization works by randomly dropping out unit activations in a network for a single gradient step. The more you drop out, the stronger the regularization.

0.0 = no dropout regularization.

1.0 = drop out all nodes. The model learns nothing.

Values between 0.0 and 1.0 = more useful.

There are also many variants of ReLU designed to address dead units, such as LeakyReLU.

KEY TERMS backpropagation; dropout regularization; exploding gradient problem; vanishing gradient problem

One versus all

One-vs.-all provides a way to use binary classification for a series of yes or no predictions across multiple possible labels. Given N possible solutions, it consists of N separate binary classifiers, one for each possible outcome.

Given a picture of a piece of fruit, four different recognizers might be trained: is this image an apple? an orange? a banana? a grape? This approach is reasonable when the total number of classes is small, but becomes **increasingly inefficient** as the number of classes rises.

A significantly more efficient one-vs.-all model uses a deep neural network in which each output node represents a different class, with a sigmoid activation function applied to the output layer.

Some real-world multi-class problems entail choosing from millions of separate classes.

One versus one, or softmax

With sigmoid on the output layer, the probabilities **don't sum to 1.0**: for the fruit example they sum to 1.43. In a one-vs.-all approach, the probability of each binary set of outcomes is determined independently of all the others.

$$p(y = j|\mathbf{x}) = \frac{e^{(\mathbf{w}_j^T \mathbf{x} + b_j)}}{\sum_{k \in K} e^{(\mathbf{w}_k^T \mathbf{x} + b_k)}}$$

Softmax assigns decimal probabilities to each class such that all probabilities add up to 1.0. This additional constraint helps training converge more quickly. The formula extends logistic regression to multiple classes.

To perform softmax, the hidden layer directly preceding the output layer, called the **softmax layer**, must have the same number of nodes as the output layer.

In the course's example, sigmoid predicts a 0.84 chance the image is a pear; softmax predicts 0.63, with all four fruits summing to 1.0.

Softmax options, and its one assumption

Full softmax calculates a probability for every possible class. It is fairly cheap when the number of classes is small but becomes prohibitively expensive when the number climbs.

Candidate sampling calculates a probability for all the positive labels but only for a random sample of negative labels. If we are determining whether an input image is a beagle or a bloodhound, we don't have to provide probabilities for every non-doggy example.

Multi-class examples: is this dog a beagle, a basset hound, or a bloodhound? Is that plane a Boeing 747, Airbus 320, Boeing 777, or Embraer 190?

Softmax assumes each example is a member of **exactly one** class. If an input image might contain a bowl of both apples and oranges, you may not use softmax; you must rely on multiple logistic regressions.

KEY TERMS binary classification; multi-class classification; one-vs.-all classification; softmax

10

Embeddings

Lower-dimensional representations of sparse data.

Embeddings 45 minutes

Imagine developing a food-recommendation application, where users input their favorite meals and the app suggests similar meals they might like. You curate a dataset of 5,000 popular meal items, including borscht, hot dog, salad, pizza, and shawarma, and create a `meal` feature that one-hot encodes each of them.

Encoding refers to the process of choosing an initial numerical representation of data to train the model on. Each one-hot vector has a length of 5,000, one entry per menu item.

Objectives.

Visualize vector representations of word embeddings, such as `word2vec`. Distinguish encoding from embedding. Describe contextual embedding.

Pitfalls of sparse data representations

- **Number of weights.** With M entries in your one-hot encoding and N nodes in the first layer, the model has to train $M \times N$ weights for that layer.
- **Number of datapoints.** The more weights, the more data you need to train effectively.
- **Amount of computation.** The more weights, the more computation required to train and use the model. It's easy to exceed the capabilities of your hardware.
- **Amount of memory.** The more weights, the more memory needed on the accelerators that train and serve it.
- **On-device ML.** To run on local devices, you need to make your model smaller and decrease the number of weights.

Embeddings are **lower-dimensional representations** of sparse data that address all of these issues.

KEY TERMS one-hot encoding; neural network; weight

Embedding space

An embedding is a vector representation of data in embedding space. Generally, a model finds potential embeddings by projecting the high-dimensional space of initial data vectors into a **lower-dimensional** space.

To give an idea, place hot dog, pizza, salad, shawarma, and borscht on a single imaginary dimension of "sandwichness". Where would an apple strudel fall? Between hot dog and shawarma, but it also has an additional dimension of "dessertness". Borscht is much more liquid than the others, which suggests a third dimension, "liquidness".

An embedding represents each item in n -dimensional space with n floating-point numbers, typically in the range -1 to 1 or 0 to 1 .

In 2D, apple strudel could be $(0.5, 0.3)$ and hot dog $(0.2, -0.5)$.

The distance between any two items can be calculated mathematically and interpreted as a measure of relative **similarity**.

Real-world embedding spaces

In the real world, embedding spaces are d -dimensional, where d is much higher than 3 though lower than the dimensionality of the data. For word embeddings, d is often 256, 512, or 1024. The individual dimensions are rarely as understandable as "dessertness" or "liquidness". Sometimes what they mean can be inferred, but **not always**.

Embeddings will usually be specific to the task. "Cereal" and "breakfast sausage" would probably be close together in the embedding space of a time-of-day model but far apart in the embedding space of a vegetarian versus non-vegetarian model.

The ML practitioner usually sets the specific task and the number of embedding dimensions. The model then tries to arrange the training examples to be close in an embedding space with that number of dimensions, or tunes for the number if d is not fixed.

Static embeddings

One task has general applicability: **predicting the context of a word**. Models trained to do so assume that words appearing in similar contexts are semantically related. Training data that includes "They rode a burro down into the Grand Canyon" and "They rode a horse down into the canyon" suggests that "horse" appears in similar contexts to "burro".

word2vec trains on a corpus of documents to obtain a single global embedding per word. When each word or data point has a single embedding vector, this is called a **static embedding**.

Research suggests that static embeddings, once trained, encode some degree of semantic information, particularly in relationships between words.

The specific embedding vectors generated depend on the corpus used for training.

word2vec refers both to an algorithm for obtaining static word embeddings and to a set of pretrained word vectors.

KEY TERMS embedding vector; embedding space

Obtaining embeddings

Dimensionality reduction. Principal component analysis has been used to create word embeddings. Given a set of instances like bag of words vectors, PCA tries to find highly correlated dimensions that can be collapsed into a single dimension.

Training an embedding as part of a neural network. Create a hidden layer of size d designated as the **embedding layer**, where d is both the number of nodes in the hidden layer and the number of dimensions in the embedding space. As in any deep network, the parameters are optimized during training to minimize loss.

For the food recommender: compile users' top five favorite foods, set four of them as feature data, and randomly set aside the fifth as the positive label the model aims to predict, optimizing with a softmax loss.

A one-hot encoding of `hot dog` might be translated into the three-dimensional embedding vector $[2.98, -0.75, 0]$.

Contextual embeddings

Words can mean different things in different contexts. "Yeah" means one thing on its own, but the opposite in the phrase "Yeah, right." "Post" can mean mail, to put in the mail, earring backing, marker at the end of a horse race, postproduction, pillar, to put up a notice, to station a guard, or after.

With static embeddings each word is a **single point** in vector space, even though it may have a variety of meanings. Contextual embeddings allow a word to be represented by multiple embeddings that incorporate information about the surrounding words as well as the word itself.

ELMo aggregates the static embedding with embeddings from other layers, which encode front-to-back and back-to-front readings of the sentence.

BERT masks part of the input sequence.

Transformers use a self-attention layer to weight the relevance of the other words in a sequence to each individual word, and add a positional embedding.

KEY TERMS bag of words; BERT; embedding layer; embedding vector; neural network; positional encoding; softmax

11

LLMs

From tokens to Transformers.

Intro to large language models 45 minutes

A language model estimates the probability of a token or sequence of tokens occurring within a longer sequence of tokens. A token could be a word, a subword, or even a single character.

When I hear rain on my roof, I _____ in my kitchen.

Probability	Token(s)
9.4%	cook soup
5.2%	warm up a kettle
3.6%	cower
2.5%	nap
2.2%	relax

An application can use the probability table to make predictions: the highest probability, or a random selection from tokens above a threshold.

By modeling the statistical patterns of tokens, modern language models develop extremely powerful internal representations of language and can **generate plausible language**.

Tokens

Most modern language models tokenize by **subwords**, chunks of text containing semantic meaning. The chunks vary in length from single characters like punctuation or the possessive s to whole words. Prefixes and suffixes might be represented as separate subwords.

- `unwatched` becomes `un`, `watch`, `ed`.
- `cats` becomes `cat`, `s`.
- `antidisestablishmentarianism` becomes `anti`, `dis`, `establish`, `ment`, `arian`, `ism`.

Tokenization is language specific, so the number of characters per token differs across languages. For English, one token corresponds to about 4 characters or about three quarters of a word, so 400 tokens is about 300 English words.

Tokens are the atomic unit of language modeling, and are now also being successfully applied to computer vision and audio generation.

N-gram language models

N-grams are ordered sequences of words used to build language models, where N is the number of words in the sequence.

```
"you are very nice"
```

```
2-grams: you are / are very / very nice
```

```
3-grams: you are very / are very nice
```

Given two words as input, a 3-gram model predicts the likelihood of the third. Given `orange is`, hundreds of 3-grams could follow. Two of them are `orange is ripe`, about the fruit, and `orange is cheerful`, about the colour.

In language models, **context** is helpful information before or after the target token. Context can help a language model determine whether `orange` refers to a citrus fruit or a colour.

Humans retain relatively long contexts. While watching Act 3 of a play, you retain knowledge of characters introduced in Act 1.

Why N-grams run out of context

The only context a 3-gram provides is the first two words. Due to lack of context, language models based on 3-grams make a lot of mistakes.

Longer N-grams provide more context. However, as N grows, the relative occurrence of each instance **decreases**. When N becomes very large, the language model typically has only a single instance of each occurrence of N tokens, which isn't very helpful in predicting the target token.

Is a 6-gram model better than a 5-gram model for English? It depends on the size and diversity of the training set. If the training set spans millions of diverse documents, the 6-gram model will probably win. Otherwise most of the 6-grams will be rare.

Recurrent neural networks train on a sequence of tokens and provide more context than N-grams. An RNN can gradually learn, and learn to ignore, selected context from each word in a sentence, kind of like you would when listening to someone speak.

RNNs evaluate information token by token, and training them for long contexts is constrained by the vanishing gradient problem.

Transformers

LLMs make much better predictions than N-gram language models or recurrent neural networks because they contain far more parameters and gather far more context. The most successful and widely used architecture for building LLMs is the **Transformer**.

A full Transformer consists of an **encoder**, which converts input text into an intermediate representation, and a **decoder**, which converts that intermediate representation into useful text. Both are enormous neural nets.

Encoder-only architectures map input text into an intermediate representation, often an embedding layer. Use cases: predicting any token in the input sequence, or creating a sophisticated embedding to serve as input for another system.

Decoder-only architectures generate new tokens from the text already generated, and typically excel at generating sequences.

Self-attention

On behalf of each token of input, self-attention asks: **how much does each other token of input affect the interpretation of this token?**

```
The animal didn't cross the street  
because it was too tired.
```

Each of the eleven words is paying attention to the other ten. The pronoun `it` typically refers to a recent noun, but which one, the animal or the street? Self-attention rates `animal` as more important. Change the final word to `wide`, and self-attention would hopefully rate `street` as more relevant instead.

The "self" refers to the input sequence. Some attention mechanisms weigh relations of input tokens to tokens in an output sequence; self-attention only weighs relations between tokens in the input sequence.

Bidirectional self-attention gathers context from words on either side; encoders use it.

Unidirectional gathers from one side only; decoders use it, because generating token by token requires it.

Multi-head, multi-layer

Each self-attention layer is typically comprised of multiple self-attention **heads**. Since the parameters of each head are initialized to random values, different heads can learn different relationships. One head might determine which noun a pronoun refers to; another might learn grammatical relevance.

A complete Transformer stacks multiple self-attention layers. Earlier layers might focus on basic syntax; deeper layers integrate that to grasp **sentiment, context, and thematic links** across the entire input.

$$O(N^2 \cdot S \cdot D)$$

N = tokens in the context

Transformers contain hundreds of billions or even trillions of parameters. This course has generally recommended fewer parameters over more, because fewer uses fewer resources. However, research shows that Transformers with more parameters **consistently outperform** those with fewer.

How LLMs are trained

The primary ingredient is a phenomenal amount of training text, typically somewhat filtered. The first phase is usually unsupervised learning on **masked predictions**: certain tokens are intentionally hidden and the model trains by trying to predict them.

```
Oranges are traditionally ___ by hand.  
Once clipped from a tree, ___ don't ripen.
```

Extensive training on enormous numbers of masked examples enables an LLM to learn that "harvested" or "picked" are high probability matches for the first token and "oranges" or "they" for the second.

An LLM is just a neural net, so loss, the number of masked tokens the model correctly considered, guides the degree to which backpropagation updates parameter values.

An optional further training step called **instruction tuning** can improve an LLM's ability to follow instructions.

You probably will never train an LLM from scratch.

How an LLM generates text

LLMs are essentially **autocomplete** mechanisms that can automatically predict thousands of tokens. You can think of a user's question to an LLM as the given sentence followed by an imaginary mask.

```
User's question: What is the easiest trick  
                  to teach a dog?
```

```
LLM's response: ---
```

An LLM trained on a massive number of mathematical word problems can give the appearance of doing sophisticated mathematical reasoning. However, those LLMs are basically just auto-completing a word problem prompt.

For "My dog, Max, knows how to perform many traditional dog tricks", an LLM generates probabilities for the masked follow-on sentence:

3.1% "For example, he can sit, stay, and roll over."

2.9% "For example, he knows how to sit, stay, and roll over."

Benefits and problems

Benefits. LLMs can generate clear, easy-to-understand text for a wide variety of target audiences, and can make predictions on tasks they are explicitly trained on. Some researchers claim LLMs can also make predictions for input they were not explicitly trained on, but other researchers have refuted this claim.

Problems in training. Gathering an enormous training set. Consuming multiple months and enormous computational resources and electricity. Solving parallelism challenges.

Problems in inference. LLMs **hallucinate**, meaning their predictions often contain mistakes. They consume enormous amounts of computational resources and electricity. Like all

Training LLMs on larger datasets typically reduces the amount of resources required for inference, though the larger training sets incur more training resources.

Fine-tuning

A foundation LLM is trained on enough natural language to know a remarkable amount about grammar, words, and idioms, and can even write poetry. But its generative text output isn't a solution for other kinds of common ML problems, such as regression or classification. For those, a foundation LLM serves as a **platform rather than a solution**.

Fine-tuning trains on examples specific to the task your application will perform. Engineers can sometimes fine-tune a foundation LLM on just a few hundred or a few thousand training examples.

Standard fine-tuning is often computationally expensive, because it updates the weight and bias of every parameter on each backpropagation iteration.

Parameter-efficient tuning adjusts only a subset.

A fine-tuned model contains the **same** number of parameters as the foundation LLM. Ten billion in, ten billion out.

Distillation

Distillation creates a smaller version of an LLM. The distilled LLM generates predictions much faster and requires fewer computational and environmental resources. However, its predictions are generally **not quite as good**.

The most common form uses bulk inference to label data. That labeled data then trains a new, smaller **student** model that can be more affordably served. The labeled data serves as a channel by which the larger **teacher** model funnels its knowledge to the smaller model.

For an online toxicity scorer, use a large offline scorer to label training data, then distill a model small enough to handle live traffic.

A teacher can funnel a numerical score instead of a binary label. A numerical score is a richer training signal, enabling the student to predict borderline classes too.

Prompt engineering

Humans learn well from examples. So do LLMs. Showing one example is **one-shot prompting**; showing several is **few-shot prompting**; showing none is **zero-shot prompting**.

```
one-shot:  peach: drupe
           apple: -----      ->  apple: pome

few-shot:  plum: drupe
           pear: pome
           lemon: ----

zero-shot: apple: -----
```

Without context, the zero-shot prompt might return information about the **technology company** rather than the fruit.

Prompt engineering doesn't alter the model's parameters. Prompts leverage the pattern-recognition abilities of the existing LLM.

Offline inference

The number of parameters in an LLM is sometimes so large that online inference is too slow to be practical for real-world tasks like regression or classification. Many engineering teams rely on **offline inference**, also known as bulk or static inference: rather than responding to queries at serving time, the trained model makes predictions in advance and caches them.

It doesn't matter if it takes a long time for an LLM to complete its task if the LLM only has to perform the task once a week or once a month.

KEY TERMS backpropagation; distillation; few-shot prompting; fine-tuning; offline inference; one shot prompting; online inference; parameter efficient tuning; prompt engineering; zero-shot prompting

Google Search used an LLM to perform offline inference in order to cache a list of over 800 synonyms for Covid vaccines in more than 50 languages. Search then used the cached list to identify queries about vaccines in live traffic.

Like any form of ML, LLMs generally share the biases of the data they were trained on and the data they were distilled on.

12

Production ML

The model is about 5 percent of the codebase.

Production ML systems 70 minutes

Real-world production ML systems are large ecosystems and the model is just a single, relatively small part. At the heart of a real-world system is the ML model code, but it often represents only **5% or less** of the total codebase.

That's not a misprint. An ML production system devotes considerable resources to the input data: collecting it, verifying it, and extracting features from it.

The other components: data collection, feature extraction, process management tools, data verification, configuration, machine resource management, monitoring, and serving infrastructure.

The ML model code part of the course's diagram is dwarfed by the other nine components.

Static versus dynamic training

	Static training (offline)	Dynamic training (online)
	Train once, serve that same model for a while	Train continuously or frequently, serve the most recent model
Advantages	Simpler. You only need to develop and test the model once.	More adaptable. Keeps up with changes to the relationship between features and labels.
Disadvantages	Sometimes staler. If the relationship changes over time, predictions degrade.	More work. You must build, test, and release a new product all the time.

KEY TERMS dynamic training; static training

If your dataset truly isn't changing over time, choose static training because it is cheaper. However, datasets tend to change, even those with features you think are as constant as sea level.

Even with static training, you must still monitor your input data for change. A flower-buying model trained on July and August data made terrible predictions around Valentine's Day.

Static versus dynamic inference

Static inference, also called offline or batch inference, means the model makes predictions on a bunch of common unlabeled examples and caches them somewhere. **Dynamic inference**, also called online or real-time inference, means the model only makes predictions on demand.

A very complex model that takes one hour to infer a prediction would be an excellent situation for static inference. If it mistakenly used dynamic inference and many clients requested predictions around the same time, most of them wouldn't receive a prediction for **hours or days**. A model that infers in 2 milliseconds can serve clients quickly and efficiently through dynamic inference.

Static inference. Don't need to worry much about cost of inference; can do post-verification before pushing. But can only serve cached predictions, and update latency is measured in hours or days.

Dynamic inference. Can infer on any new item as it comes in, great for long tail predictions. But compute intensive and latency sensitive, which may limit model complexity, and monitoring needs are more intensive.

KEY TERMS example; inference; unlabeled example

When to transform data

Before training. Write code or use specialized tools to transform the raw data, then store it where the model can ingest it. The system transforms raw data only once, and can analyze the entire dataset to determine the best transformation strategy. But you must recreate the transformations at prediction time. **Beware of training-serving skew.**

While training. The transformation is part of the model code. You can still use the same raw data files if you change the transformations, and you're ensured the same transformations at training and prediction time. But complicated transforms increase model latency, and

Training-serving skew is more dangerous with dynamic inference, because the software that transforms the raw dataset usually differs from the software that serves predictions. Systems using static inference can sometimes use the same software.

Per-batch transformation is tricky: a Z-score of -2.5 in one batch won't mean the same as -2.5 in another. Precompute the mean and standard deviation across the entire dataset and use them as constants.

Testing a deployment

If only deploying a model were as easy as pressing a big Deploy button. A full machine learning system requires tests for validating input data, validating feature engineering, validating the quality of new model versions, validating serving infrastructure, and testing integration between pipeline components.

Many software engineers favor test-driven development. However, TDD can be tricky in ML. Before training your model, you **can't write a test to validate the loss**. Instead, you must first discover the achievable loss during model development and then test new model versions against it.

Reproducible training.

Deterministically seed the random number generator. Initialize model components in a fixed order. Take the average of several runs. Use version control, even for preliminary iterations.

Even after following these guidelines, other sources of nondeterminism might still exist.

Two kinds of quality degradation

Sudden degradation. A bug in the new version could cause significantly lower quality. Validate new versions by checking their quality against the previous version.

Slow degradation. Your test for sudden degradation might not detect a slow decline over multiple versions. Ensure your model's predictions on a validation dataset meet a **fixed threshold**. If your validation dataset deviates from live data, update it and ensure the model still meets the same threshold.

If your model is updated faster than your server, it could have different software dependencies from the server. Stage the model in a sandboxed version of the server.

To test updates to API calls without retraining, write a unit test that generates random input data and runs a single step of gradient descent. If it completes without errors, then any updates to the API probably haven't ruined your model.

To run integration tests faster, train on a subset of the data or with a simpler model.

Write a data schema

To monitor your data, continuously check it against expected statistical values by writing rules that the data must satisfy. This collection of rules is a **data schema**.

- Understand the range and distribution of your features. For categorical features, understand the set of possible values.
- Encode your understanding into the schema. Ensure user-submitted ratings are always in the range 1 to 5. Check that the word "the" occurs most frequently in an English text feature.
- Test your data against the schema, to catch anomalies, unexpected values of categorical variables, and

Your model doesn't train on raw data; it trains on feature engineered data. Check that separately with unit tests: all numeric features scaled between 0 and 1, one-hot vectors containing a single 1 and $N - 1$ zeroes, Z-score means of 0, outliers handled.

Check metrics for important data slices

A successful whole sometimes obscures an unsuccessful subset. A model with great overall metrics might still make terrible predictions for certain situations.

Your unicorn model performs well overall, but performs poorly when making predictions for the Sahara desert.

If you are the kind of engineer satisfied with an overall great AUC, you might not notice. Subsets of data like this are called **data slices**. Compare model metrics for these slices against the metrics for your entire dataset.

Model metrics don't necessarily measure the real-world impact of your model. Changing a hyperparameter might increase AUC, but how did the change affect user experience?

To measure real-world impact you need separate metrics. You could survey users of your model to confirm that they really did see a unicorn when the model predicted they would.

Training-serving skew

Type	Definition	Example	Solution
Schema skew	Training and serving input data do not conform to the same schema	The format or distribution of serving data changes while the model continues to train on old data	Use the same schema to validate both; separately check statistics the schema misses, such as the fraction of missing values
Feature skew	Engineered data differs between training and serving	Feature engineering code differs between training and serving	Apply the same statistical rules across both; track the number of skewed features and the ratio of skewed examples

The Golden Rule.

Ensure that training and production **mimic each other** as closely as possible. The more closely your training task matches your prediction task, the better your ML system will perform.

Always consider what data is available to your model at prediction time.

Label leakage

Label leakage means the ground truth labels you're trying to predict have inadvertently entered your training features. Label leakage is sometimes very difficult to detect.

A cancer model using patient age, gender, prior conditions, **hospital name**, vital signs, test results, and heredity performs exceedingly well on validation and test sets and terribly in the real world. Certain hospitals specialize in treating cancer, so the model learned that patients assigned to those hospitals very likely had cancer. At inference time most patients hadn't yet been assigned to a hospital.

Another example: predicting daily revenue using the number of customers as a feature. You don't know the number of customers at prediction time, before the day's sales are complete.

When you get amazing evaluation metrics, like 0.99 AUC, look for these sorts of features that can bleed into your label.

Monitor model age and performance

If the serving data evolves with time but your model isn't retrained regularly, you will see a decline in model quality. Track the time since the model was retrained on new data and set a threshold age for alerts. Monitor the model's age **throughout the pipeline** to catch pipeline stalls.

- Track model performance by versions of code, model, and data.
- Test training steps per second against the previous version and a fixed threshold.
- Catch memory leaks by setting a threshold for memory use.
- Monitor API response times and track their percentiles.
- Monitor the number of queries answered per second.

During training, your weights and layer outputs should not be NaN or Inf. Write tests to check. Additionally, test that more than half of the outputs of a layer are not zero.

Testing quality on served data

Testing quality in serving is hard because real-world data is not always labeled. If your serving data is not labeled, consider these tests.

- Generate labels using human raters.
- Investigate models that show significant statistical bias in predictions.
- Track real-world metrics. If you're classifying spam, compare your predictions to user-reported spam.
- Serve a new model version on a **fraction** of your queries. As you validate it, gradually switch all queries to the new version.

In production, calculating a model's quality is different from calculating it during training. You won't necessarily have access to the ground truth. Write custom monitoring instrumentation to capture metrics that act as a proxy: in a mail app, monitor the percentage of mail users move to spam. If it jumps from 0.5% to 3%, that signals a potential issue.

Randomization and hashing

Make your data generation pipeline **reproducible**. For a fair experiment, your datasets should be identical except for the new feature you are testing.

Seed your random number generators, so the RNG outputs the same values in the same order each time, recreating your dataset.

Use invariant hash keys. The inputs to your hash function shouldn't change each time you run the data generation program. Don't use the current time or a random number in your hash if you want to recreate your hashes on demand.

If the hash key only used the query, then across multiple days you'd always include that query or always exclude it. That is bad: the training set sees a less diverse set of queries, and evaluation sets become artificially hard because they won't overlap with training data.

Instead hash on **query plus query date**, giving a different hashing each day. Use the date the query was logged, not the current date, so hashing stays deterministic.

KEY TERMS AUC: clipping: ground truth: hashing: label leakage: one-hot encoding: outliers: scaling: training-serving skew

Questions to ask in production

Is each feature helpful? Continuously monitor your model to remove features that contribute little or nothing. If the input data for that feature abruptly changes, your model's behavior might also abruptly change in undesirable ways. Also ask whether the usefulness of the feature justifies the **maintenance burden** of including it.

Is your data source reliable? Is the signal coming from a server that crashes under heavy load? From humans that go on vacation every August? Does the system that computes your model's input data ever change? How often, and how will you know?

It is always tempting to add more features. A new feature that makes predictions slightly better certainly seems better than slightly worse predictions; however, the extra feature adds to your maintenance burden.

Consider creating your own copy of the data you receive from the upstream process. Then only advance to the next version of the upstream data when you are certain that it is safe to do so.

Is your model part of a feedback loop?

Sometimes a model can affect its own training data: the results from some models become, directly or indirectly, input features to that same model. Sometimes a model can affect another model.

Model A is a bad predictive model for stock prices. Since Model A is buggy, it mistakenly decides to buy stock in Stock X. Those purchases **drive up the price** of Stock X. Model B uses the price of Stock X as an input feature, so Model B can come to false conclusions about its value. Model B's behavior, in turn, can affect Model A, possibly triggering a tulip mania or a slide in Company X's stock.

Susceptible. A traffic-forecasting model using beach crowd size. A book-recommendation model using popularity. A university-ranking model that rates schools by selectivity.

Not susceptible. An election-results model that forecasts after the polls have closed. A housing-value model using size, bedrooms, and location.

13

AutoML

Automating the repetitive parts of the workflow.

With manual training, you write code using an ML framework to create a model, choose which algorithms to explore, and iteratively tune hyperparameters. Two things make this heavy.

Repetitive tasks. During model development you typically need to explore different combinations of algorithms and hyperparameters. For small or exploratory projects this may not be a problem, but for larger projects these repetitive tasks can be time consuming.

Specialized skills. If a team does not have a dedicated data scientist, doing this work manually might not even be feasible.

AutoML is a process of automating certain tasks in a machine learning workflow: a set of tools and technologies that make building models faster and more accessible to a wider group of users.

The tasks: feature engineering and feature selection; identifying an appropriate ML algorithm and selecting the best hyperparameters; evaluating metrics generated during training.

AutoML tools and workflow

No-code tools typically take the form of web applications that let you configure and run experiments through a user interface. **API and CLI tools** provide advanced automation features, but require more, sometimes significantly more, programming and ML expertise.

The high level workflow steps are the same as for custom training; the main difference is that AutoML handles some tasks for you. Before importing data you still need to **label your data**, clean and format it, and perform feature transformations the tool does not support.

Real-world data tends to be messy, so expect to clean your data before using it. Even with AutoML you need to determine the best treatments for your particular dataset and problem.

Ensure that the tool you choose supports the objectives of your project, your data source, the data types in your dataset, and the size of your dataset.

Configuring an AutoML run

Import your data by specifying the data source. During import, the tool assigns a semantic data type to each data value.

Analyze and **refine**. During import, AutoML tools try to determine the correct semantic type for each feature, but these are only **guesses**. Check the types designated to all features and change them if assigned incorrectly.

Configure. Select the ML problem type. Select which column is the label. Select the set of features. Select the set of ML algorithms AutoML considers in the model search. Select the evaluation metric AutoML uses to choose the best model.

Postal codes stored as numbers would be detected as continuous numeric data. That is incorrect, and you would want to change the semantic type to categorical.

Some tools allow custom transformations. For a housing dataset with a `description` column, a `LENGTH` function could generate a new `description_length` feature.

Training may take a while to complete, on the order of hours.

Benefits of AutoML

- **Save time** by avoiding extensive manual experimentation to find the best model.
- **Improve quality.** AutoML tools can comprehensively search for the highest quality model.
- **Build without specialized skills.** A side effect of automating ML tasks is that it **democratizes** ML.
- **Smoke test a dataset.** Even for an expert, AutoML quickly gives a baseline estimate for whether a dataset has enough signal in all of its noise. If AutoML can't build even a mediocre model, it might not be worth building a good one by hand.
- **Evaluate a dataset.** Even if you don't use the resulting model, AutoML may help you determine which features aren't worth gathering.

After training, examine feature importance metrics, the architecture and hyperparameters used to build the model, and the plots and metrics collected during training.

Limitations of AutoML

- **Model quality may not be as good as manual training.** A motivated expert with enough time can often create a model with better prediction quality.
- **Model search and complexity can be opaque.** It is difficult to have insight into how the tool arrived at the best model, and models generated with AutoML are difficult to reproduce manually.
- **Multiple runs may show more variance.** Different runs may search different portions of the space and wind up in moderately, or even significantly, different places.
- **Models can't be customized during training.** If your use case requires customization or tweaking during the training process, AutoML may not be the right choice.

One thing you can count on when building a model from scratch is that you need **large amounts of data**. The advantage with AutoML is that you can mostly ignore the architecture and hyperparameter search and focus primarily on the quality of your data.

Specialized AutoML systems using transfer learning can train with significantly less data: a few hundred labeled images rather than hundreds of thousands.

Is AutoML right for your project?

*AutoML for a team with limited ML experience,
or an experienced team seeking productivity gains
with no customization requirements.*

Custom manual training is more appropriate when model quality is important and the team needs to customize their model. It may require more time for experimentation, but the team can often achieve a **higher quality** model.

KEY TERMS AutoML; transfer learning

14

Fairness

Auditing data and predictions for bias.

Evaluating a machine learning model responsibly requires doing more than just calculating overall loss metrics. Before putting a model into production, it's critical to **audit training data and evaluate predictions for bias**.

ML models are not inherently objective. Practitioners train models by feeding them a dataset of training examples, and human involvement in the provision and curation of this data can make a model's predictions susceptible to bias.

Objectives.

Become aware of common human biases that can inadvertently be reproduced by ML algorithms. Proactively explore data to identify sources of bias before training a model. Evaluate model predictions for bias.

Wikipedia's catalog of cognitive biases enumerates over 100 different types of human bias that can affect our judgment.

Reporting, historical, and automation bias

Reporting bias occurs when the frequency of events, properties, or outcomes captured in a dataset does not accurately reflect their real-world frequency. People tend to focus on documenting circumstances that are unusual or especially memorable.

Historical bias occurs when historical data reflects inequities that existed in the world at that time.

Automation bias is a tendency to favor results generated by automated systems over those generated by non-automated systems, **irrespective of the error rates of each.**

A sentiment model trained on book reviews sees mostly extreme opinions, because people were less likely to review a book they did not respond to strongly. It is then less able to predict sentiment of reviews using more subtle language.

A 1960s city housing dataset contains home-price data reflecting discriminatory lending practices in effect that decade.

A sprocket manufacturer was eager to deploy a groundbreaking tooth defect model, until the

Selection bias

Selection bias occurs if a dataset's examples are chosen in a way that is not reflective of their real-world distribution.

Coverage bias occurs if data is not selected in a representative fashion. Consumers who bought a competing product were never surveyed, so that group was not represented in the training data.

Non-response bias, also participation bias, occurs if data ends up unrepresentative due to participation gaps. Consumers who bought the competing product were 80% more likely to refuse to complete the survey.

Sampling bias occurs if proper randomization is not used. The surveyor chose the first 200 consumers who responded to an email, who might have been **more enthusiastic** than average purchasers.

All three examples come from the same scenario: a model trained to predict future sales of a new product based on phone surveys.

Group attribution bias

Group attribution bias is a tendency to generalize what is true of individuals to the entire group to which they belong.

In-group bias is a preference for members of your own group, or for characteristics that you also share. Two practitioners training a resume-screening model are predisposed to believe that applicants who attended the same computer-science academy as they both did are **more qualified**.

Out-group homogeneity bias is a tendency to stereotype individual members of a group to which you do not belong, or to see their characteristics as more uniform. The same two practitioners believe that all applicants who did not attend a computer-science academy lack sufficient expertise.

Implicit bias occurs when assumptions are made based on one's own model of thinking and personal experiences that don't necessarily apply more generally.

A practitioner training a gesture-recognition model uses a head shake as a feature to indicate "no". However, in some regions of the world, a head shake actually signifies "yes".

Confirmation and experimenter's bias

Confirmation bias occurs when model builders unconsciously process data in ways that affirm pre-existing beliefs and hypotheses.

Experimenter's bias occurs when a model builder keeps training a model until it produces a result that **aligns with their original hypothesis**.

Both are illustrated with the same practitioner: someone building a model that predicts aggressiveness in dogs, who had an unpleasant encounter with a hyperactive toy poodle as a child and ever since has associated the breed with aggression.

Under confirmation bias, the practitioner unconsciously discarded features that provided evidence of docility in smaller dogs.

Under experimenter's bias, when the trained model predicted most toy poodles to be relatively docile, the practitioner retrained the model several more times until it produced a result showing smaller poodles to be more violent.

KEY TERMS automation bias; confirmation bias; coverage bias; experimenter's bias; group attribution bias; historical bias; implicit bias; in-group bias; out-group homogeneity bias; non-response bias; reporting bias; sampling bias; selection bias

Red flags in your dataset

Missing feature values. If one or more features have missing values for a large number of examples, that could indicate that certain key characteristics of your dataset are under-represented.

Unexpected feature values. Look for examples that contain feature values that stand out as especially uncharacteristic or unusual. These could indicate problems during data collection.

Data skew. Any sort of skew, where certain groups may be under- or over-represented relative to their real-world prevalence, can introduce bias. Break out results **by subgroup** not just in aggregate

In a rescue-dog adoptability dataset, 1,500 of 5,000 examples are missing temperament values. If that correlates with breed or with age, predictions become less accurate for those groups.

Even missing for all dogs from big cities is a risk: location may serve as a proxy for physical characteristics.

A labrador retriever recorded as 35 years old is implausible: the oldest verified dog, Bluey, lived 29 years and 5 months.

Mitigating bias

Augment the training data. If an audit uncovered missing, incorrect, or skewed data, the most straightforward fix is often to collect more. The downside is that this can be infeasible, either due to a lack of available data or resource constraints.

Adjust the optimization function. Log loss does not take subgroup membership into consideration. Instead choose a function designed to penalize errors in a **fairness-aware** fashion that counteracts the imbalances you've identified.

MinDiff balances the errors for two different slices of data by adding a penalty for differences in the prediction distributions for the two groups.

Counterfactual Logit Pairing ensures that changing a sensitive attribute of a given example doesn't alter the model's prediction for that example.

The two are complementary: collect more data to reduce a discrepancy by 30%, then use MinDiff to reduce it by another 50%.

Evaluating for bias

Metrics calculated against an entire test or validation set don't always give an accurate picture of how fair the model is. Great overall performance for a majority of examples may **mask poor performance on a minority subset**. Aggregate metrics such as precision, recall, and accuracy are not necessarily going to expose these issues.

The running example: an admissions model selects 20 students from a pool of 100 candidates, belonging to two demographic groups, a majority group of 80 students and a minority group of 20 students.

There are a variety of metrics, each of which provides a different mathematical definition of fairness. The module explores three in depth:

demographic parity,
equality of opportunity,
counterfactual fairness.

Demographic parity

If the two admissions rates are equal, the model's predictions exhibit demographic parity: a student's chance of being admitted doesn't vary by demographic group.

	Majority group	Minority group
Accepted	16	4
Rejected	64	16
Acceptance rate	20%	20%

The key benefit is that both groups are represented in the admitted class in the same proportions as they are in the candidate pool.

The significant drawback: it does not take the **distribution** of predictions for each group, the number classified as qualified versus unqualified, into account when deciding how the 20 slots should be allocated.

Where demographic parity breaks

Break the same pool out by qualification as well as group. Of 35 qualified majority students, 16 were accepted. Of 15 qualified minority students, 4 were accepted.

$$\text{Majority acceptance rate} = \frac{16}{35} = 46\%$$

$$\text{Minority acceptance rate} = \frac{4}{15} = 27\%$$

Even though both groups have an overall acceptance rate of 20%, satisfying demographic parity, the rates for qualified students differ sharply.

KEY TERMS bias (ethics/fairness); demographic parity

Where the distribution of a preferred label such as "qualified" varies greatly between groups, demographic parity may not be the optimal metric for evaluating fairness.

There may be additional benefits and drawbacks not discussed here, worth considering depending on the goals the model is designed to achieve and the social context in which its predictions will be used.

Equality of opportunity

Compare the acceptance rates for just the **qualified** candidates. If those are equal, the model exhibits equality of opportunity: students with the preferred label have an equal chance of being admitted, irrespective of group.

	Majority group		Minority group	
	Accepted	Rejected	Accepted	Rejected
Qualified	14	21	6	9
Unqualified	0	45	0	5

Both qualified groups have an acceptance rate of 40%.

These predictions do **not** satisfy demographic parity: a majority student has a 17.5% chance of acceptance, a minority student 30%. But a qualified student has a 40% chance either way, arguably a fairer outcome in this use case.

The benefit: the ratio of positive to negative predictions may vary across groups, provided the model is equally successful at predicting the preferred label.

Drawbacks of equality of opportunity

It is designed for use cases where there is a **clear-cut preferred label**. If it's equally important that the model predict both the positive and the negative class for all groups, it may make sense to use **equalized odds**, which enforces equal success rates for both labels.

It also assesses fairness by comparing error rates in aggregate for demographic groups, which may not always be feasible. If the admissions dataset did not have a `demographic_group` feature, it would not be possible to break out acceptance rates and compare them.

It is possible for predictions to satisfy **both** demographic parity and equality of opportunity. In the course's worked confusion matrices, both groups have a positive prediction rate of 25% and a true positive rate of 33%.

Counterfactual fairness

Often demographic-group membership is recorded for only a small percentage of examples, for instance students who opted to self-identify. With demographic data missing for 94% of examples, neither demographic parity nor equality of opportunity is feasible.

Counterfactual fairness stipulates that two examples that are **identical in all respects**, except a given sensitive attribute, should result in the same model prediction. For the 6% of examples that do contain demographic features, compare pairs of individual predictions and see if they have been treated equitably.

Because parity and equality of opportunity assess groups in aggregate, they may mask bias issues at the level of individual predictions.

A model might accept qualified candidates from both groups in the same proportion, but reject the most qualified minority candidate while accepting the most qualified majority candidate with the exact same credentials.

Choosing a fairness metric

The key downside of counterfactual fairness is that it doesn't provide as holistic a view of bias. Identifying and remediating a handful of inequities in pairs of examples may not be sufficient to address **systemic** bias issues that affect entire subgroups.

Some definitions of fairness are even **mutually incompatible**, meaning it may be impossible to satisfy them simultaneously for a given model's predictions. So how do you choose? Consider the context in which the model is being used and the overarching goals. Is the goal equal representation, or equal opportunity?

Where possible, practitioners can do both an aggregate fairness analysis, using demographic parity or equality of opportunity, and a counterfactual fairness analysis, to gain the widest range of insights.

KEY TERMS bias (ethics/fairness); demographic parity; equality of opportunity; equalized odds; mutually incompatible

BLOCK 3

Problem Framing

Whether ML is the right approach at all, and if it is, how to state the problem in ML terms.

15

Problem framing

Analyzing a problem to isolate what has to be solved.

Problem framing 45 minutes

Problem framing is the process of analyzing a problem to isolate the individual elements that need to be addressed to solve it. It helps determine your project's technical feasibility and provides a clear set of goals and success criteria. When considering an ML solution, effective problem framing can determine whether or not your product **ultimately succeeds**.

At a high level, ML problem framing consists of two distinct steps: determining whether ML is the right approach for solving a problem, and framing the problem in ML terms.

Formal problem framing is the critical beginning for solving an ML problem, as it forces us to better understand both the problem and the data in order to design and build a bridge between them.

TensorFlow engineer

Problem framing ensures that an ML approach is a good solution to the problem **before** beginning to work with data and train a model.

State the goal, in non-ML terms

Begin by stating your goal in non-ML terms. The goal is the answer to the question, **what am I trying to accomplish?**

Application	Goal
Weather app	Calculate precipitation in six-hour increments for a geographic region
Fashion app	Generate a variety of shirt designs
Video app	Recommend useful videos
Mail app	Detect spam
Financial app	Summarize financial information from multiple news sources
Map app	Calculate travel time
Banking app	Identify fraudulent transactions
Dining app	Identify cuisine by a restaurant's menu
Ecommerce app	Reply to reviews with helpful answers

Objectives.

Identify if ML is a good solution for a problem. Learn how to frame an ML problem. Understand how to pick the right model and define success metrics.

ML is a specialized tool

Some view ML as a universal tool that can be applied to all problems. In reality, ML is a **specialized tool suitable only for particular problems**. You don't want to implement a complex ML solution when a simpler non-ML solution will work.

	Output	Training technique
Predictive ML	Makes a prediction, for example classifying an email as spam, guessing tomorrow's rainfall, or predicting a stock price. The output can typically be verified against reality.	Typically uses lots of data to train a supervised, unsupervised, or reinforcement learning model to perform a specific task.

Both take text, image, audio, video, or numerical input.

A generative model can then be used for tasks that can be framed as fill-in-the-blank tasks, or fine-tuned by training on labeled data for a specific task, like classification.

Benchmark against a non-ML solution

To confirm that ML is the right approach, first verify that your current non-ML solution is optimized. If you don't have one, try solving the problem manually using a heuristic. The non-ML solution is the **benchmark** you'll use to determine whether ML is a good use case.

Quality. How much better do you think an ML solution can be? If you think it might be only a small improvement, that might indicate the current solution is the best one.

Cost and maintenance. How expensive is the ML solution in both the short and long term?

Can the ML solution justify the increase in cost? Small improvements in large systems can easily justify the cost.

How much maintenance will the solution require? In many cases, ML implementations need dedicated long-term maintenance.

Does your product have the resources to support training or hiring people with ML expertise?

Six characteristics of usable data

- **Abundant.** The more relevant and useful examples in your dataset, the better your model will be.
- **Consistent and reliable.** An ML-based weather model benefits from data gathered over many years from the same reliable instruments.
- **Trusted.** Will the data come from trusted sources you control, like logs from your product, or from sources you have little insight into, like the output of another ML system?
- **Available.** Make sure all inputs are available at prediction time in the correct format. If a feature value will be difficult to obtain then, **omit that feature.**
- **Correct.** Some labels will inevitably be wrong, but if more than a small percentage are, the model will produce poor predictions.
- **Representative.** The datasets should accurately reflect the events,

Data is the driving force of predictive ML. If you can't get the data you need in the required format, your model will make poor predictions.

Predictive power

The more correlated a feature is with a label, the more likely it is to predict it. In a weather dataset, `cloud_coverage`, `temperature`, and `dew_point` would be better predictors of rain than `moon_phase` or `day_of_week`.

Determining which features have predictive power can be a time consuming process. You can explore it manually by removing and adding a feature while training. You can automate it using algorithms such as Pearson correlation, adjusted mutual information, and Shapley value, which provide a numerical assessment.

For the video app, you could hypothesize that `video_description`, `length`, and `views` might be good predictors of which videos a user would want to watch.

Three key attributes to look for when analyzing your datasets: representative of the real world, contains correct values, and features have predictive power for the label.

Predictions versus actions

*There's no value in predicting something
if you can't turn the prediction into an **action**
that helps users.*

A model that predicts whether a user will find a video useful should feed into an app that recommends useful videos. A model that predicts whether it will rain should feed into a weather app.

KEY TERMS dataset; feature; heuristic; label; prediction

16

ML framing

Ideal outcome, model output, success metrics.

Ideal outcome and the model's goal

App	Ideal outcome	Model's goal
Weather	Calculate precipitation in six hour increments for a region	Predict six-hour precipitation amounts for specific regions
Fashion	Generate a variety of shirt designs	Generate three varieties of a shirt design from text and an image
Video	Recommend useful videos	Predict whether a user will click on a video
Mail	Detect spam	Predict whether or not an email is spam
Map	Calculate travel time	Predict how long it will take to travel between two points
Banking	Identify fraudulent transactions	Predict if a transaction was made by the card holder

Independent of the ML model, what's the ideal outcome? What is the exact task you want your product or feature to perform? This is the same statement you defined when you stated the goal.

Then **connect** the model's goal to the ideal outcome by explicitly defining what you want the model to do.

Identify the output you need

If you need to classify something or make a numeric prediction, you'll probably use **predictive ML**. If you need to generate new content or produce output related to natural language understanding, you'll probably use **generative AI**.

ML system		Example output
Classification	Binary	Classify an email as spam or not spam
	Multiclass single-label	Classify an animal in an image
	Multiclass multi-label	Classify all the animals in an image
Numerical	Unidimensional regression	Predict the number of views a video will get
	Multidimensional regression	Predict blood pressure

Generative AI outputs. Text: summarize an article, reply to reviews, translate documents, write product descriptions, analyze legal documents. Image: marketing images, visual effects, product design variations. Audio: dialogue in a specific accent, a short musical composition. Video: realistic-looking videos. Multimodal: a video with text captions.

Classification or regression: the caching example

You want to cache videos based on predicted popularity: 50 or more views uses the expensive cache, 30 to 50 the cheap cache, fewer than 30 no cache. A regression model seems right, since you're predicting a number.

But when training it, you realize it produces the **same loss** for a prediction of 28 and 32 on a video that got 30 views. Your app will behave very differently for 28 versus 32, but the model considers both predictions equally good.

A classification model would produce a higher loss for 28 than 32. In a sense, classification models produce thresholds by default.

Regression models are unaware of product-defined thresholds. If your app's behavior changes significantly because of small differences in a regression model's predictions, consider a classification model instead.

Two lessons from the caching example

Predict the decision. When possible, predict the decision your app will take. In the video example, a classification model would predict the decision if the categories it classified videos into were "no cache", "cheap cache", and "expensive cache". **Hiding your app's behavior from the model can cause your app to produce the wrong behavior.**

Understand the problem's constraints. If your app takes different actions based on different thresholds, determine if those thresholds are fixed or dynamic.

Dynamic thresholds: use a regression model and set the threshold limits in your app's code. This lets you easily update the thresholds while still having the model make reasonable predictions.

Fixed thresholds: use a classification model and label your datasets based on the threshold limits.

Most cache provisioning is dynamic, so for that problem regression is the best choice. For many problems the thresholds are

Proxy labels

In the weather app the label directly addresses the ideal outcome: `precipitation_amount`. In the video app the ideal outcome is to recommend useful videos, but there's no label called `useful_to_user`. Proxy labels substitute for labels that aren't in the dataset.

Proxy label	Issue
User will click "like"	Most users never click "like"
Video will be popular	Not personalized
User will share the video	Some users don't share; sometimes people share videos because they dislike them
User will click play	Maximizes clickbait
How long they watch	Favors long videos

A workable proxy label for usefulness might be `shared` OR `liked`.

No proxy label can be a perfect substitute for your ideal outcome. All will have potential problems. Pick the one that has the least problems for your use case.

Generation: three ways to customize

In most cases you won't train your own generative model, because that requires massive amounts of training data and computational resources. Instead you'll customize a pre-trained one.

Distillation creates a smaller version of a larger model: you generate a synthetic labeled dataset from the larger model and use it to train the smaller one.

Fine-tuning or parameter-efficient tuning further trains the model on a dataset containing examples of the type of output you want.

Prompt engineering tells the model the task you want it to do, or shows illustrative examples with the desired outputs.

```
Translate words from English  
to Spanish.
```

```
English: Car  
Spanish: Auto
```

```
English: Airplane  
Spanish: Avion
```

```
English: Home  
Spanish: -----
```

Distillation and fine-tuning update the model's parameters. Prompt engineering **does not**.

Success metrics

Success metrics define what you care about, like engagement or helping users take appropriate action. They **differ from the model's evaluation metrics**, like accuracy, precision, recall, or AUC.

Weather app

Success	Users open the "Will it rain?" feature 50 percent more often than before
Failure	Users open the feature no more often than before

Video app

Success	Users spend on average 20 percent more time on the site
Failure	Users spend on average no more time on site than before

Define ambitious success metrics. High ambitions can cause gaps between success and failure. The undefined gap is not what's important.

What's important is your model's capacity to move closer to, or exceed, the definition of success.

Would improving the model get you closer to your defined success criteria? A model might have great evaluation metrics but not move you closer, indicating that even a perfect model wouldn't meet your criteria.

Is the model worth improving?

- **Not good enough, but continue.** The model shouldn't be used in production, but over time it might be significantly improved.
- **Good enough, and continue.** The model could be used in production, and it might be further improved.
- **Good enough, but can't be made better.** The model is in production, but it is probably as good as it can be.
- **Not good enough, and never will be.** The model shouldn't be used in production and no amount of training will probably get it there.

When deciding to improve the model, re-evaluate if the increase in

After defining the success and failure metrics, determine how often you'll measure them: six days, six weeks, or six months after implementing the system.

When analyzing failure metrics, determine **why** the system failed. The model might be predicting which videos users will click, but start recommending clickbait titles that cause engagement to drop off.

17

Implementing

Start simple, then decide whether complexity is justified.

Start simple

Most of the work in ML is on the **data side**, so getting a full pipeline running for a complex model is harder than iterating on the model itself. After setting up your data pipeline and implementing a simple model that uses a few features, you can iterate on creating a better model.

Simple models provide a good baseline, even if you don't end up launching them. In fact, using a simple model is probably better than you think. Starting simple helps you determine whether or not a complex model is **even justified**.

Trained models only really work when the label and features match your dataset exactly. If a trained model uses 25 features and your dataset includes only 24 of them, it will most likely make bad predictions.

Commonly, practitioners use matching subsections of inputs from a trained model for fine-tuning or transfer learning.

If your solution is a generative AI model, you'll almost always fine-tune a pre-trained model instead of training your own.

Monitoring, decided during framing

Model deployment. A newly trained model might be worse than the model currently in production. If it is, you'll want to prevent it from being released and get an alert that your automated deployment has failed.

Training-serving skew. If any incoming features used for inference have values outside the distribution range of the training data, you'll want to be alerted, because the model will likely make poor predictions.

Inference server. If you're providing inferences through an RPC system, monitor the RPC server itself and get an alert if it stops providing inferences.

If your model was trained to predict temperatures for equatorial cities at sea level, your serving system should alert you of incoming data with latitudes, longitudes, or altitudes **outside the range** the model was trained on.

Conversely, it should alert you if the model is making predictions outside the distribution range seen during training.

The two steps, in six moves

Verify that ML is a good approach.

- Understand the problem.
- Identify a clear use case.
- Understand the data.

Frame the problem in ML terms.

- Define the ideal outcome and the model's goal.
- Identify the model's output.
- Define success metrics.

These steps save time and resources by setting **clear goals** and providing a shared framework for working with other ML practitioners.

When implementing ML solutions, always follow Google's Responsible AI Principles. For a hands-on introduction to improving fairness and mitigating bias, see the ML fairness module in block 2.

BLOCK 4

Managing ML Projects

From an idea to a production-ready implementation: the phases, the team, the plan, and the pipelines that keep a model fresh.

Why traditional ML, in a generative age

This course focuses on traditional ML models. Although generative AI is in the spotlight, traditional ML plays a vital role at Google, underpinning many services and projects: predicting travel times in Maps, estimating the price of airline tickets in Flights, predicting compute quota for Cloud customers, recommending relevant videos in YouTube.

In general, the principles for managing traditional ML projects are **identical** for managing generative AI projects. Where there's a significant difference, the course calls it out.

Objectives.

Define the phases and elements of an ML project. Describe how to plan and manage one. Determine business and model success metrics. Recognize the iterative process of running ML experiments. Design a solution for productionizing ML pipelines. Implement responsible ML and AI practices at each development phase.

18

Phases

Four phases, each with a goal, tasks, and an outcome.

The four phases

Phase	Goal	Outcome
Ideation and planning	Determine if ML is the best solution to your problem	A design doc outlining how to solve a problem with an ML solution
Experimentation	Build a model that solves the business problem	A model with good enough quality to put into production
Pipeline building and production-ization	Build and implement the infrastructure for scaling, monitoring, and maintaining models in production	Validated ML pipelines

A clear understanding of the phases helps establish engineering responsibilities, manage stakeholder expectations, and efficiently allocate resources.

Successfully moving through the phases, **often iteratively**, is foundational for designing, assembling, and building ML models that solve business problems over the long term.

What each phase is underestimated for

Ideation and planning. It can take a long time to understand the data and metrics required for a production-ready system.

Experimentation. New-to-ML practitioners often underestimate the challenges of designing and implementing the appropriate experimentation **tooling and processes**. It's not uncommon to try hundreds of experiments before finding the right combination of features, hyperparameters, and model architecture.

Productionization. It's easy to underestimate the complexity of productionizing data pipelines and evaluating models, especially as features evolve. Not only do you have to implement all the monitoring infrastructure required for a

Multiple challenges exist at each phase. Not realizing, and planning for, them may lead to missed deadlines, frustrated engineers, and failed projects.

Before spending time drafting a design doc or writing code, you should first verify that ML is the right solution to your problem.

19

Team

Six roles, and the process docs that align them.

The six common roles

Role	Knowledge and skills	Main deliverable
ML product manager	Deep understanding of ML strengths, weaknesses, and the development process; aligns business problems to ML solutions	Product requirements document
Engineering manager	Sets, communicates, and achieves team priorities	Design docs, project plans, performance evaluations
Data scientist	Quantitative and statistical analysis; identifies and tests features, prototypes models	Reports and data visualizations

Successful ML projects have teams with each role well represented. In smaller teams, individuals will need to handle the responsibilities for **multiple roles**.

Data engineers are responsible for

Process documentation

Because the roles, tools, and frameworks vary widely in ML development, it's critical to establish common practices through excellent process documentation. Good process docs answer questions like:

- How is the data generated for the model? How do we examine, validate, and visualize it?
- How do we modify an input feature or label in the training data?
- How do we customize the data generation, training, and evaluation pipeline?
- How do we obtain testing examples? What metrics will we use to judge model quality?
- How do we launch our models in production? How will we know if something is wrong with our model?

One engineer might think that just getting the right data is sufficient to begin training, while a more responsible engineer will validate that the dataset is anonymized correctly and document its metadata and provenance.

Making sure engineers share common definitions for processes and design patterns reduces confusion and **increases the team's velocity**.

Everyone is right from their own perspective

What constitutes ML best practices can differ between companies, teams, and individuals. Some team members might consider experimental notebooks the main deliverable, while others will want to work in R. Some have a passion for software engineering, someone else thinks monitoring is the most important thing, yet someone else is aware of good feature productionization practices but wants to use Scala.

Everyone is right from their own perspective and if steered correctly, the mix will be a **powerhouse**. If not, it can be a mess.

Establishing the tools, processes, and infrastructure the team will use **before writing a line of code** can be the difference between the project failing after two years or successfully launching a quarter ahead of schedule.

Performance evaluations under uncertainty

*A team member's performance should not be directly connected to the **success of the project**.*

It's not uncommon for team members to spend weeks investigating solutions that are ultimately unsuccessful. Even in these cases, their high-quality code, thorough documentation, and effective collaboration should contribute positively toward their evaluation. Due to the ambiguity and uncertainty inherent in ML, people managers need to set clear expectations and define deliverables early.

Stakeholders and deliverables

As early as possible, define your project's stakeholders, the expected deliverables, and the preferred communication methods.

Design doc. Before you write a line of code, create a doc that explains the problem, the proposed solution, the potential approaches, and possible risks. It functions as a way to receive feedback and address concerns.

Experimental results. The record of your experiments with their hyperparameters and metrics, the training stack, and saved versions of your model at certain checkpoints.

Production-ready implementation. A full pipeline for

Some stakeholders might assume that ML is similar to traditional software engineering with **deterministic outcomes**. They might not understand why the project's progress is stalled or why a project's milestones are non-linear.

To manage expectations, be clear about the complexities, timeframes, and deliverables at each stage.

20

Feasibility

Data, difficulty, quality, technical requirements, cost.

Data availability

While ML might be a theoretically good solution, you still need to estimate its real-world feasibility. A solution might technically work but be **impractical or impossible to implement**.

Quantity. Can you get enough high-quality data? Are labeled examples scarce, difficult to get, or too expensive? Getting labeled medical images or translations of rare languages is notoriously hard.

Feature availability at serving time. Teams have spent considerable amounts of time training models only to realize that some features didn't become available until days after the model required them.

A model predicts whether a customer will click a URL, using `user_age` as a training feature. When the model serves a prediction, `user_age` isn't available, perhaps because the user hasn't created an account yet.

Classification models require numerous examples for **every** label. If the training dataset contains limited examples for some labels, the model can't make

Data for generative AI

Pre-trained generative AI models often require curated datasets to excel at domain-specific tasks.

Technique	Number of required examples
Zero-shot prompting	0
Few-shot prompting	tens to hundreds
Parameter efficient tuning	hundreds to tens of thousands
Fine-tuning	thousands to tens of thousands, or more

Parameter efficient tuning covers LoRA and prompt tuning. The quality of the fine-tuning data is more important than the quantity.

Once pre-trained, generative AI models have a **fixed knowledge base**. If content in the model’s domain changes often, you’ll need a strategy to keep it up to date: fine-tuning, retrieval-augmented generation, or periodic pre-training.

Problem difficulty

What initially appears to be a plausible approach might actually turn out to be an open research question. What seems practical and doable might turn out to be unrealistic.

Has a similar problem already been solved? If so, it's likely you'll be able to use parts of their model to build yours. If you're attempting a new problem, expect to invest substantial amounts of time and resources: you're basically doing a research project that doesn't have a guaranteed solution.

Is the nature of the problem difficult? Humans classify the type of animal in an image at about 95% accuracy, and handwritten digits at about 99% accuracy. So a model to classify animals is more difficult than one to classify

Are there potentially bad actors?

If people will be actively trying to exploit your model, you'll be in a constant race to update it before it can be misused. Spam filters can't catch new types of spam when someone exploits the model to create emails that appear legitimate.

Generative AI vulnerabilities

Input source. Where will the input come from? Can adversarial prompts leak training data, preamble material, database content, or tool information?

Output use. How will outputs be used? Will the model output raw content or will there be intermediary steps that test and verify it's appropriate? Providing raw output to plugins can cause a number of security issues.

Fine-tuning. Fine-tuning with a **corrupted dataset** can negatively affect the model's weights, causing it to output incorrect, toxic, or biased content.

Fine-tuning requires a dataset that's been verified to contain high-quality examples.

Prediction quality and its cost

The required prediction quality depends on the type of prediction. Recommending the wrong video might create a bad user experience. Wrongly flagging a video as violating a platform's policies might generate support costs, or worse, legal fees.

Generally, the higher the required prediction quality, the harder the problem. Projects often reach diminishing returns as you try to improve quality. Increasing a model's precision from 99.9% to 99.99% could mean a **10-times increase** in the project's cost, if not more.

Generative AI output.

Although generative models produce fluent and coherent content, it's not guaranteed to be factual. False statements from generative AI models are called **confabulations**. Many use cases still require human verification before production use, for example LLM-generated code.

What are the legal, financial, or ethical consequences of biased, plagiarized, or toxic content?

Technical requirements

- **Latency.** How fast do predictions need to be served?
- **Queries per second.** What are the QPS requirements?
- **RAM usage.** For training and for serving.
- **Platform.** Online, in a web browser, on a phone or tablet, or offline with predictions saved in a table?
- **Interpretability.** Will your product need to answer questions like "why was a specific piece of content marked as spam"?
- **Retraining frequency.** When the underlying data changes rapidly, frequent or continuous retraining may be necessary, at significant cost.

Explaining ML predictions is hard and an active area of research. Tools like Model Cards and the What-if Tool can help you begin to understand a model's behavior and design for interpretability.

For generative AI, output size affects latency more than input size. Models can parallelize their inputs, but they can only generate outputs sequentially.

Cost

Most ML projects won't be approved if the ML solution is more expensive to implement and maintain than the money it generates or saves.

Human costs. How many people will it take to go from proof of concept to production? ML projects require more people to deploy and maintain a production-ready system than to create a prototype.

Machine costs. Training, deploying, and maintaining models requires lots of compute and memory. You might need TPU quota, data pipeline infrastructure, paid data labeling, or data licensing fees.

Encountering issues related to any of these topics can make implementing an ML solution a challenge, but **tight deadlines amplify** the challenges.

Plan and budget enough time based on the perceived difficulty of the problem, and then reserve even more overhead time than you might for a non-ML project.

21

Planning

Non-linear work, planned with an experimental mindset.

Project uncertainty

Early-stage planning is difficult because the best approach typically isn't apparent when beginning a project. This inherent uncertainty makes estimating timelines hard.

In a Kaggle competition, the first few weeks drew 350 teams, who raised benchmark prediction quality from 35% to 65%. Over the next two weeks the number of teams grew from 350 to 1400. The best model achieved **68%**. Over a thousand teams, each experimenting with a variety of data transformations, architectures, and hyperparameters, only achieved producing a model with 68% prediction quality.

An example from industry illustrates the non-linearity, where the output isn't proportional to the inputs. Two teams took several months to train a model to 90% prediction quality. However, it took several teams **more than five years** to get the model ready for production with 99.9% prediction quality.

Production-ready ML is an exploratory process, requiring both a scientific and an engineering mindset.

Software engineering plans versus ML plans

In a typical software engineering plan, you define the requirements, outline the components, estimate the effort, and schedule the work. There's a **clearly defined path** to a solution. Engineers often know with a high degree of certainty the tasks they need to complete, and can estimate the work based on similar projects.

For most ML projects, you'll find the best solution by experimenting with multiple approaches in a trial-and-error process. The optimal architecture might be a simple linear model, or a neural net, or possibly a decision tree. Only by trying each approach can you discover the best solution.

By definition, experiments aren't guaranteed to be successful. Because of this, it's best to plan using an experimental framework.

Only by attempting to solve the problem can you get a better sense for the amount of time and resources a solution might require.

Three planning strategies

Time box the work. Allocate two weeks to determine if you can get access to the right kind of data. If you can, designate two more weeks to see if a simple model indicates feasibility. If a simple model fails, designate two more to try a neural net.

Scope down the requirements. If an ML solution appears promising but isn't a critical feature, plan a very simple solution this quarter and improve it iteratively in subsequent quarters.

Intern or new hire project. Directing an intern or a new hire to attempt an ML solution can be a good way to begin exploring a new space with unknown outcomes.

At the end of each timeframe, you'll have more information to determine if continuing to apply resources to the problem is worthwhile.

With any strategy, it's wise to **fail fast**. Attempt approaches with the lowest costs, but potentially the highest payoff, first. If the approach works, you've found a good solution. If it doesn't, you haven't wasted lots of time and resources.

Planning as a dynamic document

Learning to plan multiple ML approaches probabilistically takes time and experience. Your project plan might require frequent updates. Think of it as a dynamic document in **constant evolution**.

- Estimate the cost and chance of success for each approach.
- Attempt a portfolio of approaches.
- Identify lessons learned and try to improve the system one thing at a time.
- Plan for failures.

When an approach leads to a dead end, rather than being discouraged, team members understand that's part of the process. By discussing the inherent uncertainty with stakeholders, you're able to set more realistic expectations.

Occasionally an early approach leads to a breakthrough. Someone might discover a bug in the data generation pipeline or the training-validation split.

22

Success

Business metrics first, model metrics second.

Business metrics

Business metrics are the most important. They're the reason you're using ML: **you want to improve the business**. Start with quantifiable product or business metrics, as granular and focused as possible.

- Reduce a datacenter's monthly electric costs by 30 percent.
- Increase revenue from product recommendations by 12 percent.
- Increase click-through rate by 9 percent.
- Increase customer sentiment from opt-in surveys by 20 percent.

If you're not tracking the metric you want to improve, start by implementing the infrastructure to do so. Setting a goal to increase click-through rate 15% isn't logical if you're not currently measuring click-through rates.

The metric you care about might change. You might optimize revenue from recommendations, see revenue increase but subscriptions decrease, and reevaluate.

Model metrics

Determine a single metric to optimize. Classification models can be evaluated against a variety of metrics. Choosing the best model is challenging when different metrics favor different models, so agree on **one** metric to evaluate models against.

Determine acceptability goals to meet. These are different from evaluation metrics. They refer to goals a model needs to meet to be considered acceptable for an intended use case, for example "incorrect output is less than 0.1%" or "recall for the top five categories is greater than 97%".

A binary classification model detecting fraudulent transactions might optimize **recall**, correctly identifying fraud most of the time, while holding **precision** at or above a particular value as its acceptability goal.

Evaluating generative AI output presents unique challenges. For open-ended or creative output, it's more difficult than evaluating traditional ML outputs.

Model metrics do not guarantee business metrics

A team develops a churn model with 95% prediction quality and tests it on a small sample of users. Revenue doesn't increase. Customer churn actually **increases**. Possible explanations:

- Predictions don't occur early enough to be actionable. The model can only predict churn within a seven-day timeframe, which isn't soon enough to offer incentives.
- Incomplete features. Other factors contributing to churn weren't in the training dataset.
- Threshold isn't high enough. The model might need 97% or higher to be useful.

Typically teams deploy the model to **1% of users** and then monitor the business metric.

Perform early user testing to prove, and understand, the connection between the model's metrics and the business metrics.

Many skilled engineers can create models with impressive metrics. Training a good-enough model isn't typically the issue. An ML project can be destined for failure from a misalignment between business metrics and model metrics.

23

Experiments

Testable, reproducible hypotheses.

Running experiments

Determine baseline performance. The baseline acts as a measuring stick to compare experiments against. The current non-ML solution can provide the first baseline. If none exists, create a model with a simple architecture and a few features.

Make single, small changes. One change at a time, to the hyperparameters, architecture, or features. If the change improves the model, that model's metrics become the **new baseline**.

Record the progress of the experiments. Experiments with poor or neutral prediction quality are still useful to track: they signal which approaches won't work.

Examples of a single, small change: include feature X; use 0.5 dropout on the first hidden layer; take the log transform of feature Y; change the learning rate to 0.001.

Because each full training on a real-world dataset can take hours or days, consider running multiple independent experiments concurrently to explore the space quickly.

Noise in experimental results

You might encounter noise in experimental results that isn't from changes to the model or the data, making it difficult to determine if a change actually improved the model.

- **Data shuffling.** The order in which the data is presented can affect performance.
- **Variable initialization.** How the model's variables are initialized can also affect it.
- **Asynchronous parallelism.** The order in which different parts of the model are updated can affect it.
- **Small evaluation sets.** If the evaluation set is too small, it may not be representative, producing uneven

Your team should have a clear understanding of what exactly an experiment is, with a defined set of practices and artifacts: what the artifacts are, what coding practices apply, and what the standards are for reproducibility and tracking.

Wrong predictions, and stalled projects

No real-world model is perfect. Begin thinking early about how to deal with wrong predictions. A best-practices strategy encourages users to correctly label them: mail apps capture misclassified email by logging the mail users move into their spam folder, and the reverse.

Strategic fix for a stalled project. You might need to **reframe** the problem. Maybe you initially wanted linear regression to predict a numeric value, but the data wasn't good enough. Further analysis might reveal the problem can be solved by predicting whether an example is above or below a specific value, reframing it as binary classification.

Technical fix. Spend time diagnosing and analyzing wrong predictions. Isolate a few and diagnose the model's behavior in those instances. You might uncover problems with the architecture or the data.

Maybe no solution is possible. Time-box your approach, stopping if you haven't made progress within the timeframe. However, if you have a strong problem statement, then it probably warrants a solution.

KEY TERMS baseline

24

Pipelines

The goal is not one model. It is pipelines.

Why pipelines

*In production ML, the goal isn't to build a single model and deploy it.
The goal is to build **automated pipelines** for developing,
testing, and deploying models over time.*

As the world changes, trends in data shift, causing models in production to go stale. Without pipelines, replacing a stale model is an error-prone process: someone has to manually collect and process new data, train a new model, validate its quality, and finally deploy it.

The four pipelines

Serving pipeline. Delivers predictions. It exposes your model to the real world, receiving and processing the user's data, making a prediction, and delivering it.

Data pipeline. Processes user data to create training and test datasets.

Training pipeline. Trains models using the new training datasets from the data pipeline.

Validation pipeline. Validates the trained model by comparing it with the production model using test datasets generated by the data pipeline.

How they keep a fresh model in production.

A model goes into production and the serving pipeline starts delivering predictions. The data pipeline immediately begins collecting data. Based on a schedule or trigger, the training and validation pipelines train and validate a new model. When validation confirms the new model isn't worse than the production model, the new model gets deployed. This process **repeats continuously**.

Model staleness and training frequency

Models tend to go stale almost immediately after they go into production. Their training datasets captured the state of the world a day ago, or in some cases, an hour ago. Inevitably the world has changed.

Some models go stale faster than others. Models that recommend clothes typically go stale quickly because consumer preferences are notorious for changing frequently. Models that identify flowers might **never** go stale: a flower's identifying characteristics remain stable.

A recommended best practice is to train and deploy new models on a **daily** basis. Just like regular software projects that have a daily build and release process.

Establish a training frequency that reflects the nature of your data. If the data is dynamic, train often. If it's less dynamic, you might not need to train that often.

Train models **before** they go stale. Early training provides a buffer to resolve potential issues, for example if the data or training pipeline fails, or the model quality is poor.

Quick to go stale: spam, clothes recommendations, fraud detection. Not: bird species classification.

Online and offline predictions

Online predictions happen in real time, typically by sending a request to an online server and returning a prediction.

Offline predictions are precomputed and cached. To serve a prediction, the app finds the cached prediction in the database and returns it. A subscription service might predict the churn rate for every subscriber and cache it; when the app needs it, for instance to incentive users about to churn, it just looks up the precomputed prediction.

Predictions typically get **post-processed** before delivery: to remove toxic or biased content, boost more authoritative content, present a diversity of results, demote clickbait, or remove results for legal reasons.

The infrastructure is different for online and offline predictions. This course refers to both as the serving pipeline.

The feature engineering step is typically built **within** the model and not a separate, stand-alone process. The data processing code in the serving pipeline is often nearly identical to that in the data pipeline.

Logging predictions and capturing ground truth

The serving pipeline should incorporate a repository to log model predictions and, if possible, the ground truth. By aggregating predictions, you can monitor the general quality of your model and determine if it's starting to lose quality. Generally, the production model's predictions should have the **same average** as the labels from the training dataset.

In some cases the ground truth only becomes available much later. If a weather app predicts the weather six weeks into the future, the ground truth won't be available for six weeks.

A mail app implicitly captures user feedback when users move mail from their inbox to their spam folder. However, this only works when users correctly categorize their mail. When users leave spam in their inbox, because they know it's spam and never open it, that mail gets labeled "not spam" when it should be "spam".

Always try to find ways to capture and record the ground truth, but be aware of the shortcomings in feedback mechanisms.

Versioned repositories

Benefit	What it buys you
Reproducibility	Recreate and standardize model training environments and compare prediction quality among different models
Compliance	Adhere to regulatory requirements for auditability and transparency
Retention	Set data retention values for how long to store the data
Access management	Manage who can access your data through fine-grained permissions
Data integrity	Track and understand changes to datasets over time, making it easier to diagnose issues
Discoverability	Make it easy for others to find your datasets, features, and models

Model repositories additionally give you tracking and evaluation of models in production, an easy **model release process** to review, approve, release, or roll back, and reproducibility for debugging by tracing a model's datasets and dependencies across deployments.

Use **data cards** to structure information about datasets you share, and **model cards** to document a model's purpose, architecture, hardware requirements, and evaluation

Six challenges building pipelines

- **Getting access to the data you need.** You might need to explain how the data will be used and clarify how personal identifiable information issues will be solved.
- **Getting the right features.** The features used in experimentation might not be available from the real-time data.
- **Understanding how the data is collected and represented.** Don't use data you're not confident in to train a model that might go to production.
- **Understanding the tradeoffs between effort, cost, and model quality.** A new feature might require a lot of effort and only slightly improve quality, or be easy to add but prohibitively expensive to store.
- **Getting compute.** TPU quota can be hard to get, and managing

Catching these problems during **experimentation** saves time. You don't want to develop the best features and model only to learn they're not viable in production.

25

Productionization

Compute, monitoring, and the deployment runbook.

Provisioning compute

Running ML pipelines requires compute resources: RAM, CPUs, and GPUs or TPUs. Depending on your use case, you might train and serve on different hardware. In general, it's common to **train on bigger hardware and serve on smaller**.

- Can you train on less expensive hardware?
- Would switching to different hardware boost performance?
- What size is the model and which hardware will optimize its performance?
- What hardware is ideal based on your model's architecture?

Switching hardware might make the model cheaper to run, but the engineering effort to migrate might outweigh the savings.

You might allocate quota that pipelines **share**. In such cases, verify you have enough to run all your pipelines, and set up monitoring and alerting to prevent a single, errant pipeline from consuming all the quota.

When estimating quota, factor in not only production pipelines but also ongoing experiments.

What to monitor, per pipeline

Pipeline	Monitor
Serving	Skews or drifts in the serving data compared to the training data; skews or drifts in predictions; data type issues like missing or corrupted values; quota usage; model quality metrics
Data	Skews and drifts in feature values; skews and drifts in label values; data type issues; quota usage rate; quota limit about to be reached
Training	Training time; training failures; quota usage
Validation	Skew or drift in the test datasets

Also monitor **latency**, how long it takes to deliver a prediction, and **outages**, whether the model has stopped delivering predictions at all.

Good logging and monitoring practices help proactively identify issues and mitigate potential business impact. When issues do occur, alerts notify your team, and comprehensive logs facilitate diagnosing the root cause.

Comparing the **changes** in proxy metrics is more insightful than their raw numbers.

Deploying a model

Document the following before you need them.

- Approvals required to begin deployment and increase the roll out.
- How to put a model into production.
- Where the model gets deployed, for example whether there are staging or canary environments.
- What to do if a deployment fails.
- How to rollback a model already in production.

Typically you deploy new models to a **subset of users** to check that the model is behaving as expected. If it is, continue. If it's not, rollback and begin diagnosing.

After automating model training, automate validation and deployment. Automating deployments distributes responsibility and reduces the likelihood of a deployment being bottlenecked by a single person. It also reduces potential mistakes, increases efficiency and reliability, and enables on-call rotations and SRE support.

26

Ethics

Fairness, privacy, transparency, safety.

Your projects should benefit society

ML has the potential to transform society in many meaningful ways, either positively or negatively. It's critical to consider the ethical implications of your models and the systems they're a part of.

Your ML projects **should benefit society**. They shouldn't cause harm or be susceptible to misuse. They shouldn't perpetuate, reinforce, or exacerbate biases or prejudices. They shouldn't collect or use personal data irresponsibly.

Beyond adhering to responsible AI principles, aim to develop systems that incorporate fairness, privacy, transparency, and safety.

Fairness

Models exhibit bias when their training data doesn't reflect the real-world population of their users, preserves biased decisions or outcomes, or uses features with more predictive power for certain groups.

- Identify underrepresented groups in evaluation datasets. You might need to oversample a subgroup to increase their presence.
- Use golden datasets to validate the model against fairness issues and detect implicit bias.
- Avoid including sensitive features, like gender or ethnicity.
- Avoid including features with little empirical or explanatory power, especially in sensitive contexts.
- Measure potential adverse impact on particular groups, and consider intentional bias correction techniques.

In a model for approving home loans, **don't include names** in the training data. Not only is an applicant's name irrelevant to the prediction task, but leaving such an irrelevant feature in the dataset also has the potential to create implicit bias or allocative harms. The model might correlate male names with a higher probability for repayment, or vice versa.

The first step is verifying the training data accurately reflects the distribution of your users.

Privacy, transparency, safety

Privacy. Incorporate privacy design principles from the beginning. Be aware of and adhere to the European Union's Digital Markets Act for consent to share or use personal data, and EU GDPR laws. Remove all personally identifiable information from datasets and confirm your model and data repositories have the right permissions, for example **not world-readable**.

Transparency. Be accountable to people. Make it easy for others to understand what your model does, how it does it, and why it does it.

Safety. Design models to operate safely in adversarial conditions. Test your model with potential hostile inputs and

Model cards provide a template to document your model and create transparency artifacts.

Teams typically use specially designed datasets to test their models with inputs or conditions that caused the model to fail in the past.

Always consider the broader social contexts your models operate within.

Tools change; the nature of the problem does not

ML development requires using a variety of constantly evolving tools and frameworks. New ML tools continue to emerge as ways to handle complex data types, advances in hardware, and techniques for orchestrating pipelines continue to develop.

As a result, companies, organizations, and teams implement ML solutions using different tools and frameworks, which likely change over time. While common frameworks and best practices are emerging, keep in mind that the nature of your particular problem might require **custom solutions** in certain cases.

That is why this track spends its time on the phases, the data, the metrics, and the pipelines rather than on any one framework.

References

- 1 Google Developers. *Foundational Courses: Machine Learning*. Course hub.
<https://developers.google.com/machine-learning/foundational-courses>. 2025.
- 2 Google Developers. *Introduction to Machine Learning*.
<https://developers.google.com/machine-learning/intro-to-ml>. 2025.
- 3 Google Developers. *Machine Learning Crash Course*.
<https://developers.google.com/machine-learning/crash-course>. 2025.
- 4 Kevin A. Hallgren. “Computing Inter-Rater Reliability for Observational Data: An Overview and
- Information Processing Systems. Vol. 28. 2015, pp. 2503–2511.
- 9 Shachar Kaufman, Saharon Rosset, and Claudia Perlich. “Leakage in Data Mining: Formulation, Detection, and Avoidance”. In: *Proceedings of the 17th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 2011, pp. 556–563.
- 10 Moritz Hardt, Eric Price, and Nathan Srebro. “Equality of Opportunity in Supervised Learning”. In: *Advances in Neural Information Processing Systems*. Vol. 29. 2016.

Data is your *most valuable* resource

Prof. Dr.-Ing. Mark Schutera | DHBW Ravensburg